Computer Architecture

Tomasulo's Algorithm

College of Computer Science
Chongqing University

Review: Reducing CPI (or Increasing IPC)

$$CPI = CPI_{ideal} + stalls_{structural} + stalls_{dataHazard} + stalls_{control}$$

technique	reduces
forwarding/ bypassing	potential data-hazard stalls
delayed branches	control-hazard stalls
basic dynamic scheduling (scoreboarding)	data-hazard stalls from true dependencies
dynamic scheduling with register renaming	data-hazard, anti-dep. & output dep. stalls
dynamic branch prediction	control stalls
issuing multiple instruction per clock cycle	ideal CPI
speculation	data-hazard and control-hazard stalls
dynamic memory disambiguation	data-hazard stalls with memory
loop unrolling	control hazard stalls
basic compiler pipeline scheduling	data-hazard stalls
compiler dependency analysis	ideal CPI, data-hazard stalls
software pipelining & trace scheduling	ideal CPI, data-hazard stalls
compiler speculation	ideal CPI, data-hazard stalls

Review: Dynamic HW Scheduling

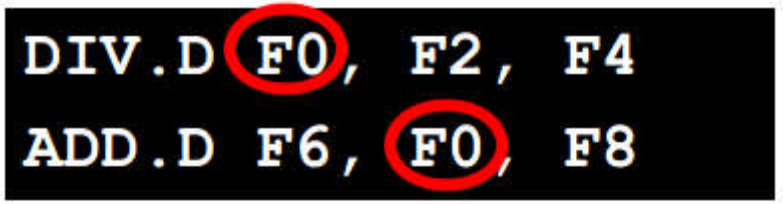


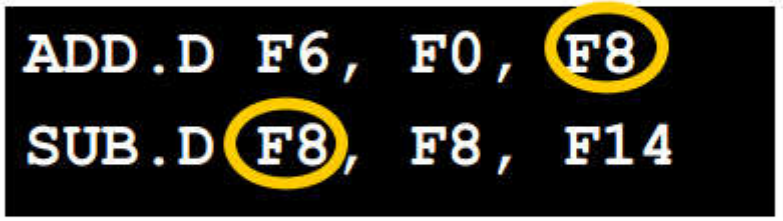
- SUB.D does not depend on the previous instructions but it can not execute because ADD.D is stalling due to a RAW hazard

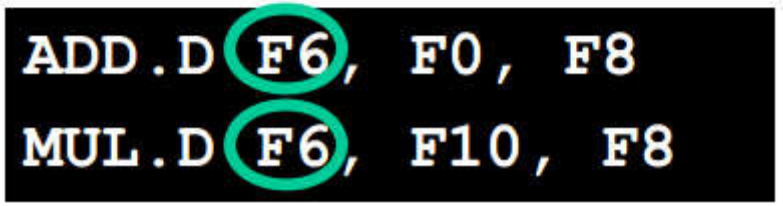
- With dynamic scheduling, the HW rearranges the instruction execution to reduce the stalls while maintaining data flow and exception behavior
  - it simplifies compiling
    - code compiled for a given pipeline can be run efficiently on another
  - it handles cases where dependencies are unknown at compile time (due to memory references)
  - it is the basis for hardware speculation

Review: Name Dependences and the New Hazards due to Out-of-Order Execution

- True data dependency: an instruction produces a value for a following instruction
  - may lead to a RAW hazard
- Name dependence: no real value must be transmitted between two instructions, but they both use the same register or memory location
  - anti-dependence
    - may lead to a WAR hazard
  - output dependence
    - may lead to a WAW hazard

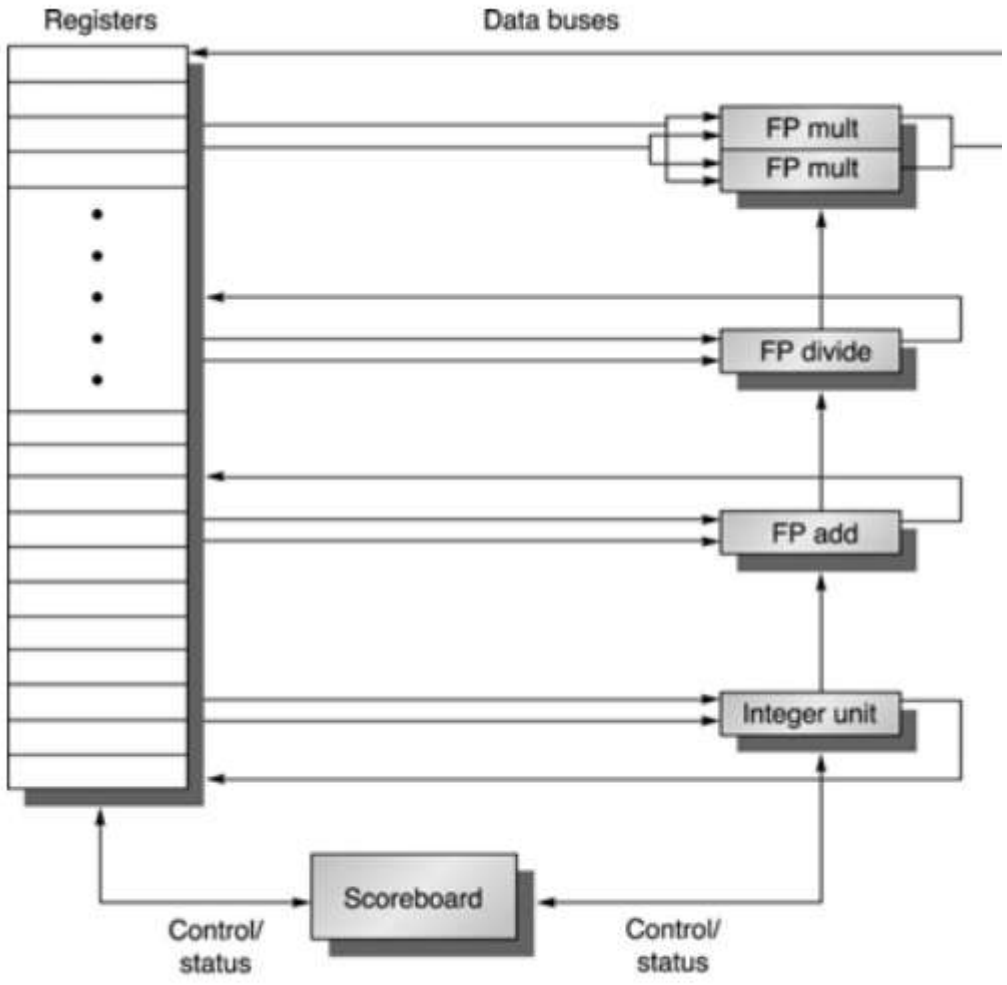




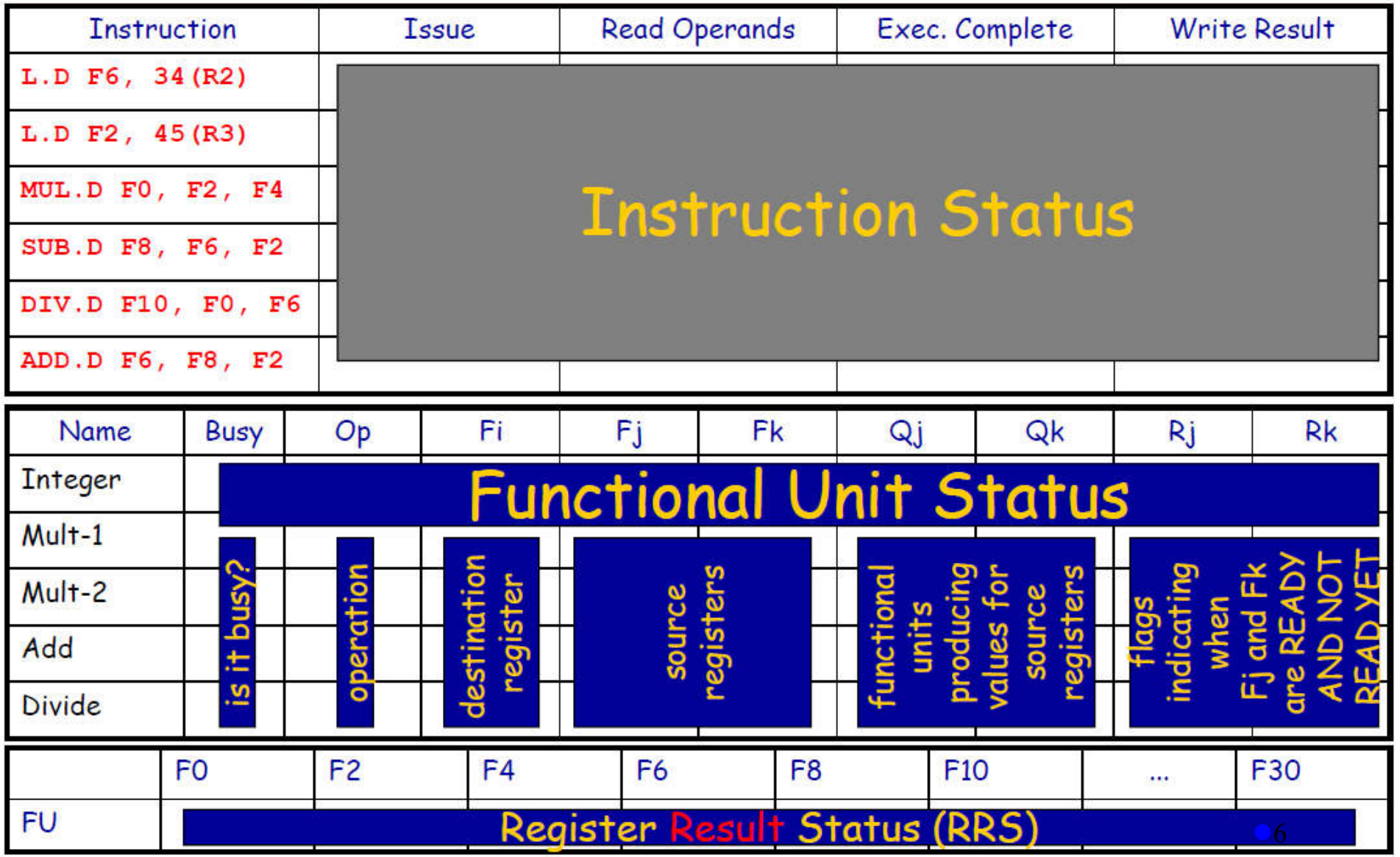


Review: Dynamic Scheduling with Scoreboard

- Goal
  - when possible, execute instructions following a stall
- Strategy
  - in-order issuing
  - out-of-order execution
  - out-of-order completion
- Implementation
  - centralized control
  - read operands together when they are all avaialable
- Hazard handling
  - Structural and WAW
    - stall issuing
  - RAW
    - solved dynamically
  - WAR
    - stall committing



MIPS Scoreboard Components

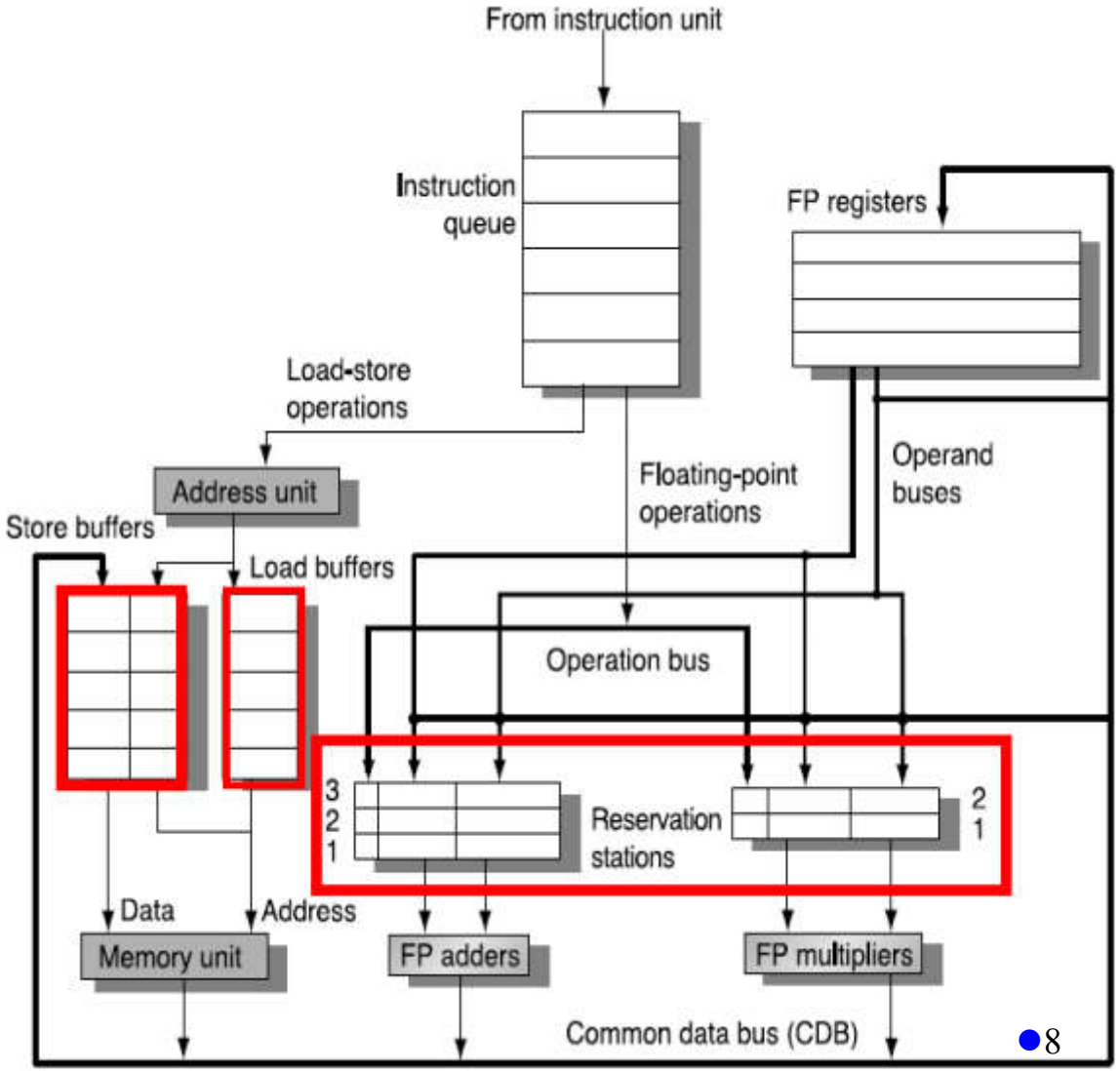


Dynamic Scheduling with Tomasulo's Algorithm

- Motivations
  - need for improving performance of floating-point unit for the whole IBM 360 family (without specialized compilers)
  - IBM 360 had only 4 DP FP registers (limiting the effectiveness of compiler scheduling)
  - IBM 360 had long memory access and long FP delays
- Main features
  - distributed control and buffering with reservation stations
  - dynamically replace register specification in instruction with value or pointer to functional unit producing the value (register renaming)
    - handle efficiently WARs and WAWs
  - tracking when operands are available to handle RAWs
  - more reservation stations than registers
    - eliminates name dependencies that compiler can't eliminate
- Basic concepts reused in many other architectures
  - Alpha 21264, Pentium II, III, IV, PowerPC, MIPS R10000, AMD K5

Assumption: Basic Architecture of MIPS FP Unit with Tomasulo's Algorithm

- Distributed control based on reservation station tag fields
- Register file bypassed in broadcasting to reservation stations via Common Data Bus (CDB)
- Precise exception
  - stall issuing operations when there is a pending branch in the pipeline
    - speculation will remove this restriction
- IBM 360 had pipelined functional units
  - but we describe it with multiple functional units



MIPS Pipeline Stages with Tomasulo's Algorithm

- Distributed hazard detection and resolution
- In-order issuing, out-of-order execution and commit
- ID, EX, and WB stages replaced by three steps that can take an arbitrary number of clock cycles

1. Issue (ID-1)
   - get new instruction from queue
   - check for availability of matching reservation station (structural hazards)
   - check for operands in the register file and read them if available (register renaming handles WAR and WAW hazards)
   - if operands are not available, keep track of functional units that will produce them
2. Execution (EX)
   - monitor CDB for missing operands
   - start execution when operands are available in reservation stations (RAW hazards)
3. Write Result (WB)
   - use CDB to broadcast the result to register file and reservation stations (including store buffers to write memory)

Reservation Station Components

Op: Operation to perform in the unit (e.g., + or –)

Vj, Vk: Value of Source operands

- Store buffers has V field, result to be stored

Qj, Qk: Reservation stations producing source registers (value to be written)

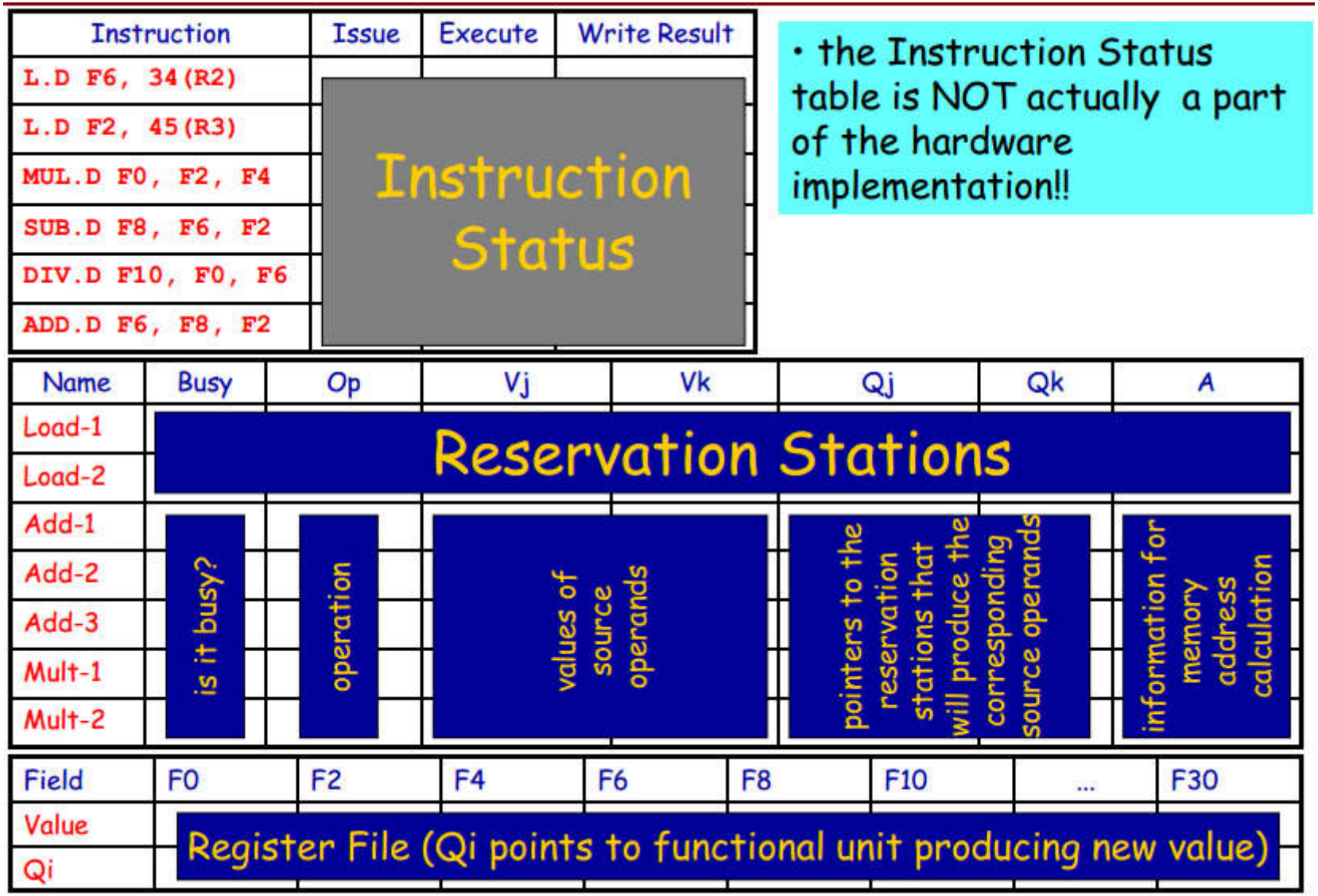
- Note: No ready flags as in Scoreboard; Qj,Qk=0 => ready
- Store buffers only have Qi for RS producing result

A: to hold memory address for load and store instructoins

Busy: Indicates reservation station or FU is busy

Register result status—Indicates which functional unit will write each register, if one exists. Blank when no pending instructions that will write that register.

Tomasulo's Algorithm Data Structure

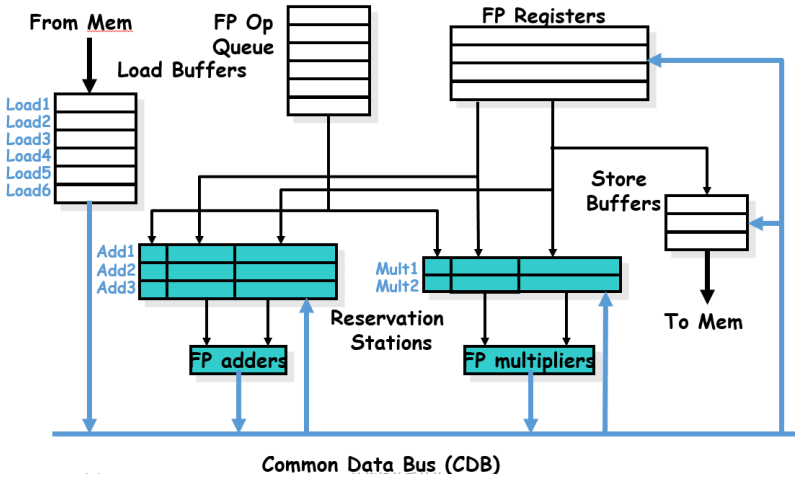


Tomasulo's Algorithm: Checking & Bookkeeping per Instruction (op D,S1,S2)

Inst. Status	Wait until	Bookkeeping
1. Issue	(RS[r].busy == no) where r is the reservation station for the functional unit matching op	if (Regs[S1].Qi ≠0) RS.[r].Qj ← Regs[S1].Qi else RS.[r].Vj ← Regs[S1].value if (Regs[S2].Qi ≠0) RS.[r].Qk ← Regs[S2].Qi else RS.[r].Vk ← Regs[S2].value RS[r].busy ← yes ; Regs[D].Qi ← r;
2. Execution	(RS.[r].Qj == 0) and (RS.[r].Qk == 0)	execute operation using operands in RS[r].Vj and RS[r].Vk and producing res
3. Write Result	RS.[r] is done & CDB is available	∀x (if (Regs[x].Qi == r) Regs[x].val ← res; Regs[x].Qi ← 0) ∀f (if (RS[f].Qj == r) RS[f].Vj ← res; RS[f].Qj ← 0) ∀f (if (RS[f].Qk == r) RS[f].Vk ← res; RS[f].Qk ← 0) RS[r].busy ← no

Tomasulo's Algorithm: Checking & Execution of Memory Accesses--Issue

Load: rs,基地址寄存器； rt : 目标寄存器存器；
Store: rs,基地址寄存器； rt : 源寄存器存器；
RS: 保留站数据结构；r:保留站编号
RegS：寄存器状态表
Reg： 寄存器堆



Load or Store:

Wait until: Buffer **r** empty

Action or bookkeeping:

if(RegS [rs].Qi≠0) ；基址寄存器未就绪（值还依赖其它指令）
{RS[r].Qj ← RegS [rs].Qi}
else {RS[r].Vj ←Reg[rs]; RS[r].Qj ←0 } ； 等待基址寄存器生成
RS[r].A ←imm; RS[r].Busy ← yes; ； 获得指令中的偏移量

Load only: RegS[rt].Qi = r; (寄存器重命名) ； 目标寄存器数据源设置为保留站r

Store only:

if(RegS[rt].Qi≠0) {RS[r].Qk ← RegS[rt].Qi} ； 回写数据源设为生成rt的保留站
else {RS[r].Vk ←Reg[rt]; RS[r].Qk←0 } ；否则直接读取回写数据

Tomasulo's Algorithm: Execution of Memory Accesses

Loads and Stores executed *in program order* based on a two-step process

1. compute the effective address when base register is available
2. then
   - if load execute as soon as the memory unit is available
   - if store, wait for the value to be stored and send it to memory unit

Inst. Status	Wait until	Bookkeeping
2. Execution	(RS.[r].Qj == 0) and r being head of load/store queue	RS[r].A ← RS[r].Vj + RS[r].A if (op is load) do res ← Mem[RS[r].A]
3. Write Result	RS.[r] is done & CDB is available if (op is store) RS.[r] is done & wait for RS[r].Qk=0	if (op is load) same steps as before... if (op is store) do Mem[RS[r].A] ← RS[r].Vk RS[r].busy ← no

Tomasulo's Algorithm Ex.: Clock Cycle = 1

Instruction	Issue	Execute	Write Result
L.D F6, 34(R2)	1		
L.D F2, 45(R3)			
MUL.D F0, F2, F4			
SUB.D F8, F6, F2			
DIV.D F10, F0, F6			
ADD.D F6, F8, F2			

- issuing first load instruction

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	yes	L.D					34+R[R2]
Load-2	no						
Add-1	no						
Add-2	no						
Add-3	no						
Mult-1	no						
Mult-2	no						

Field	F0	F2	F4	F6	F8	F10	...	F30
Value								
Qi				Load-1				

Tomasulo's Algorithm Ex.: Clock Cycle = 2

Instruction	Issue	Execute	Write Result
L.D F6, 34(R2)	1	2	
L.D F2, 45(R3)	2		
MUL.D F0, F2, F4			
SUB.D F8, F6, F2			
DIV.D F10, F0, F6			
ADD.D F6, F8, F2			

- executing first load instr.
- issuing second load instr.
- differently from CDC 6600, multiple loads can be outstanding

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	yes	L.D					34+R[R2]
Load-2	yes	L.D					45+R[R3]
Add-1	no						
Add-2	no						
Add-3	no						
Mult-1	no						
Mult-2	no						

Field	F0	F2	F4	F6	F8	F10	...	F30
Value								
Qi		Load-2		Load-1				

Tomasulo's Algorithm Ex.: Clock Cycle = 3

Instruction	Issue	Execute	Write Result
L.D F6, 34(R2)	1	2	3
L.D F2, 45(R3)	2	3	
MUL.D F0, F2, F4	3		
SUB.D F8, F6, F2			
DIV.D F10, F0, F6			
ADD.D F6, F8, F2			

- completing first load instr.
- executing second load instr.
- issuing multiplication,
- register F4 read asap and Mult-1 reservation station stores its value

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	no						
Load-2	yes	L.D					45+R[R3]
Add-1	no						
Add-2	no						
Add-3	no						
Mult-1	yes	MUL.D		R[F4]	Load-2		
Mult-2	no						

Field	F0	F2	F4	F6	F8	F10	...	F30
Value				M[34(R2)]				
Qi	Mult-1	Load-2						

Tomasulo's Algorithm Ex.: Clock Cycle = 4

Instruction	Issue	Execute	Write Result
L.D F6, 34(R2)	1	2	3
L.D F2, 45(R3)	2	3	4
MUL.D F0, F2, F4	3		
SUB.D F8, F6, F2	4		
DIV.D F10, F0, F6			
ADD.D F6, F8, F2			

- issuing subtraction
- completing second load, whose result is broadcasted to Add-1 and Mult-1
- stalling multiplication
- reservation station Vj gets value loaded by second load

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	no						
Load-2	no						
Add-1	yes	SUB.D	M[45(R3)]	M[34(R2)]			
Add-2	no						
Add-3	no						
Mult-1	yes	MUL.D	M[45(R3)]	R[F4]			
Mult-2	no						

Field	F0	F2	F4	F6	F8	F10	...	F30
Value		M[45(R3)]		M[34(R2)]				
Qi	Mult-1				Add-1			

Tomasulo's Algorithm Ex.: Clock Cycle = 5

Instruction	Issue	Execute	Write Result
L.D F6, 34(R2)	1	2	3
L.D F2, 45(R3)	2	3	4
MUL.D F0, F2, F4	3	starting from 5	
SUB.D F8, F6, F2	4	starting from 5	
DIV.D F10, F0, F6	5		
ADD.D F6, F8, F2			

- executing multiplication and subtraction (with 10- and 2-cycle execution latency, respectively)
- issuing division

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	no						
Load-2	no						
Add-1	yes	SUB.D	M[45(R3)]	M[34(R2)]			
Add-2	no						
Add-3	no						
Mult-1	yes	MUL.D	M[45(R3)]	R[F4]			
Mult-2	yes	DIV.D		M[34(R2)]	Mult-1		

Field	F0	F2	F4	F6	F8	F10	...	F30
Value		M[45(R3)]		M[34(R2)]				
Qi	Mult-1				Add-1	Mult-2		

Tomasulo's Algorithm Ex.: Clock Cycle = 6

Instruction	Issue	Execute	Write Result
L.D F6, 34(R2)	1	2	3
L.D F2, 45(R3)	2	3	4
MUL.D F0, F2, F4	3	starting from 5	
SUB.D F8, F6, F2	4	6	
DIV.D F10, F0, F6	5		
ADD.D F6, F8, F2	6		

- completed execution of subtraction
- executing multiplication (still 8 cycles to go)
- stalling division (waiting for multiplication)
- issuing addition

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	no						
Load-2	no						
Add-1	yes	SUB.D	M[45(R3)]	M[34(R2)]			
Add-2	yes	ADD.D		M[45(R3)]	Add-1		
Add-3	no						
Mult-1	yes	MUL.D	M[45(R3)]	R[F4]			
Mult-2	yes	DIV.D		M[34(R2)]	Mult-1		

Field	F0	F2	F4	F6	F8	F10	...	F30
Value		M[45(R3)]		M[34(R2)]				
Qi	Mult-1			Add-2	Add-1	Mult-2		

Tomasulo's Algorithm Ex.: Clock Cycle = 7

Instruction	Issue	Execute	Write Result
L.D F6, 34(R2)	1	2	3
L.D F2, 45(R3)	2	3	4
MUL.D F0, F2, F4	3	starting from 5	
SUB.D F8, F6, F2	4	6	7
DIV.D F10, F0, F6	5		
ADD.D F6, F8, F2	6		

- committing subtraction, broadcasting result to ADD-2
- executing multiplication (still 7 cycles to go)
- stalling division (waiting for multiplication)

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	no						
Load-2	no						
Add-1	no						
Add-2	yes	ADD.D	res{SUB.D}	M[45(R3)]			
Add-3	no						
Mult-1	yes	MUL.D	M[45(R3)]	R[F4]			
Mult-2	yes	DIV.D		M[34(R2)]	Mult-1		

Field	F0	F2	F4	F6	F8	F10	...	F30
Value		M[45(R3)]		M[34(R2)]	res{SUB.D}			
Qi	Mult-1			Add-2		Mult-2		

Tomasulo's Algorithm Ex.: Clock Cycle = 8

Instruction	Issue	Execute	Write Result
L.D F6, 34(R2)	1	2	3
L.D F2, 45(R3)	2	3	4
MUL.D F0, F2, F4	3	starting from 5	
SUB.D F8, F6, F2	4	6	7
DIV.D F10, F0, F6	5		
ADD.D F6, F8, F2	6	starting from 8	

- executing multiplication (still 6 cycles to go)
- stalling division (waiting for multiplication)
- executing addition (1 more cycle to go); register renaming avoids WAR hazard!

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	no						
Load-2	no						
Add-1	no						
Add-2	yes	ADD.D	res{SUB.D}	M[45(R3)]			
Add-3	no						
Mult-1	yes	MUL.D	M[45(R3)]	R[F4]			
Mult-2	yes	DIV.D		M[34(R2)]	Mult-1		

Field	F0	F2	F4	F6	F8	F10	...	F30
Value		M[45(R3)]		M[34(R2)]	res{SUB.D}			
Qi	Mult-1			Add-2		Mult-2		

Tomasulo's Algorithm Ex.: Clock Cycle = 9

Instruction	Issue	Execute	Write Result
L.D F6, 34(R2)	1	2	3
L.D F2, 45(R3)	2	3	4
MUL.D F0, F2, F4	3	starting from 5	
SUB.D F8, F6, F2	4	6	7
DIV.D F10, F0, F6	5		
ADD.D F6, F8, F2	6	9	

- executing multiplication (still 5 cycles to go)
- stalling division (waiting for multiplication)
- completed addition execution

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	no						
Load-2	no						
Add-1	no						
Add-2	yes	ADD.D	res{SUB.D}	M[45(R3)]			
Add-3	no						
Mult-1	yes	MUL.D	M[45(R3)]	R[F4]			
Mult-2	yes	DIV.D		M[34(R2)]	Mult-1		

Field	F0	F2	F4	F6	F8	F10	...	F30
Value		M[45(R3)]		M[34(R2)]	res{SUB.D}			
Qi	Mult-1			Add-2		Mult-2		

Tomasulo's Algorithm Ex.: Clock Cycle = 10

Instruction	Issue	Execute	Write Result
L.D F6, 34(R2)	1	2	3
L.D F2, 45(R3)	2	3	4
MUL.D F0, F2, F4	3	starting from 5	
SUB.D F8, F6, F2	4	6	7
DIV.D F10, F0, F6	5		
ADD.D F6, F8, F2	6	9	10

- executing multiplication (still 4 cycles to go)
- stalling division (waiting for multiplication)
- committed addition (without WAR hazard with previous division)

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	no						
Load-2	no						
Add-1	no						
Add-2	no						
Add-3	no						
Mult-1	yes	MUL.D	M[45(R3)]	R[F4]			
Mult-2	yes	DIV.D		M[34(R2)]	Mult-1		

Field	F0	F2	F4	F6	F8	F10	...	F30
Value		M[45(R3)]		res{ADD.D}	res{SUB.D}			
Qi	Mult-1					Mult-2		

Tomasulo's Algorithm Ex.: Clock Cycle = 11

Instruction	Issue	Execute	Write Result
L.D F6, 34(R2)	1	2	3
L.D F2, 45(R3)	2	3	4
MUL.D F0, F2, F4	3	starting from 5	
SUB.D F8, F6, F2	4	6	7
DIV.D F10, F0, F6	5		
ADD.D F6, F8, F2	6	9	10

- executing multiplication (still 3 cycles to go)
- stalling division (waiting for multiplication)

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	no						
Load-2	no						
Add-1	no						
Add-2	no						
Add-3	no						
Mult-1	yes	MUL.D	M[45(R3)]	R[F4]			
Mult-2	yes	DIV.D		M[34(R2)]	Mult-1		

Field	F0	F2	F4	F6	F8	F10	...	F30
Value		M[45(R3)]		res{ADD.D}	res{SUB.D}			
Qi	Mult-1					Mult-2		

Tomasulo's Algorithm Ex.: Clock Cycle = 12

Instruction	Issue	Execute	Write Result
L.D F6, 34(R2)	1	2	3
L.D F2, 45(R3)	2	3	4
MUL.D F0, F2, F4	3	starting from 5	
SUB.D F8, F6, F2	4	6	7
DIV.D F10, F0, F6	5		
ADD.D F6, F8, F2	6	9	10

- executing multiplication (still 2 cycles to go)
- stalling division (waiting for multiplication)

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	no						
Load-2	no						
Add-1	no						
Add-2	no						
Add-3	no						
Mult-1	yes	MUL.D	M[45(R3)]	R[F4]			
Mult-2	yes	DIV.D		M[34(R2)]	Mult-1		

Field	F0	F2	F4	F6	F8	F10	...	F30
Value		M[45(R3)]		res{ADD.D}	res{SUB.D}			
Qi	Mult-1					Mult-2		

Tomasulo's Algorithm Ex.: Clock Cycle = 13

Instruction	Issue	Execute	Write Result
L.D F6, 34(R2)	1	2	3
L.D F2, 45(R3)	2	3	4
MUL.D F0, F2, F4	3	starting from 5	
SUB.D F8, F6, F2	4	6	7
DIV.D F10, F0, F6	5		
ADD.D F6, F8, F2	6	9	10

- executing multiplication (still 1 cycle to go)
- stalling division (waiting for multiplication)

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	no						
Load-2	no						
Add-1	no						
Add-2	no						
Add-3	no						
Mult-1	yes	MUL.D	M[45(R3)]	R[F4]			
Mult-2	yes	DIV.D		M[34(R2)]	Mult-1		

Field	F0	F2	F4	F6	F8	F10	...	F30
Value		M[45(R3)]		res{ADD.D}	res{SUB.D}			
Qi	Mult-1					Mult-2		

Tomasulo's Algorithm Ex.: Clock Cycle = 14

Instruction	Issue	Execute	Write Result
L.D F6, 34(R2)	1	2	3
L.D F2, 45(R3)	2	3	4
MUL.D F0, F2, F4	3	14	
SUB.D F8, F6, F2	4	6	7
DIV.D F10, F0, F6	5		
ADD.D F6, F8, F2	6	9	10

- completed execution of multiplication
- stalling division (waiting for multiplication)

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	no						
Load-2	no						
Add-1	no						
Add-2	no						
Add-3	no						
Mult-1	yes	MUL.D	M[45(R3)]	R[F4]			
Mult-2	yes	DIV.D		M[34(R2)]	Mult-1		

Field	F0	F2	F4	F6	F8	F10	...	F30
Value		M[45(R3)]		res{ADD.D}	res{SUB.D}			
Qi	Mult-1					Mult-2		

Tomasulo's Algorithm Ex.: Clock Cycle = 15

Instruction	Issue	Execute	Write Result
L.D F6, 34(R2)	1	2	3
L.D F2, 45(R3)	2	3	4
MUL.D F0, F2, F4	3	14	15
SUB.D F8, F6, F2	4	6	7
DIV.D F10, F0, F6	5		
ADD.D F6, F8, F2	6	9	10

- committing multiplication and broadcasting the result to division, which at the next cycle can finally start the execution

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	no						
Load-2	no						
Add-1	no						
Add-2	no						
Add-3	no						
Mult-1	no						
Mult-2	yes	DIV.D	res{MUL.D}	M[34(R2)]			

Field	F0	F2	F4	F6	F8	F10	...	F30
Value	res{MUL.D}	M[45(R3)]		res{ADD.D}	res{SUB.D}			
Qi						Mult-2		

Tomasulo's Algorithm Ex.: Clock Cycle = 16

Instruction	Issue	Execute	Write Result
L.D F6, 34(R2)	1	2	3
L.D F2, 45(R3)	2	3	4
MUL.D F0, F2, F4	3	14	15
SUB.D F8, F6, F2	4	6	7
DIV.D F10, F0, F6	5	starting from 16	
ADD.D F6, F8, F2	6	9	10

• starting division execution, it will take 40 cycles

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	no						
Load-2	no						
Add-1	no						
Add-2	no						
Add-3	no						
Mult-1	no						
Mult-2	yes	DIV.D	res{MUL.D}	M[34(R2)]			

Field	F0	F2	F4	F6	F8	F10	...	F30
Value	res{MUL.D}	M[45(R3)]		res{ADD.D}	res{SUB.D}			
Qi						Mult-2		

Tomasulo's Algorithm Ex.: Clock Cycle = 55

Instruction	Issue	Execute	Write Result
L.D F6, 34(R2)	1	2	3
L.D F2, 45(R3)	2	3	4
MUL.D F0, F2, F4	3	14	15
SUB.D F8, F6, F2	4	6	7
DIV.D F10, F0, F6	5	55	
ADD.D F6, F8, F2	6	9	10

• completed division execution

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	no						
Load-2	no						
Add-1	no						
Add-2	no						
Add-3	no						
Mult-1	no						
Mult-2	yes	DIV.D	res{MUL.D}	M[34(R2)]			

Field	F0	F2	F4	F6	F8	F10	...	F30
Value	res{MUL.D}	M[45(R3)]		res{ADD.D}	res{SUB.D}			
Qi						Mult-2		

Tomasulo's Algorithm Ex.: Clock Cycle = 56

Instruction	Issue	Execute	Write Result
L.D F6, 34(R2)	1	2	3
L.D F2, 45(R3)	2	3	4
MUL.D F0, F2, F4	3	14	15
SUB.D F8, F6, F2	4	6	7
DIV.D F10, F0, F6	5	55	56
ADD.D F6, F8, F2	6	9	10

- committing division

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	no						
Load-2	no						
Add-1	no						
Add-2	no						
Add-3	no						
Mult-1	no						
Mult-2	no						

Field	F0	F2	F4	F6	F8	F10	...	F30
Value	res{MUL.D}	M[45(R3)]		res{ADD.D}	res{SUB.D}	res{DIV.D}		
Qi								

Recalling…
Scoreboard Example: Clock Cycle = 62

Instruction	Issue	Read Operands	Exec. Complete	Write Result
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5	6	7	8
MUL.D F0, F2, F4	6	9	19	20
SUB.D F8, F6, F2	7	9	11	12
DIV.D F10, F0, F6	8	21	61	62
ADD.D F6, F8, F2	13	14	16	22

Name	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	no								
Mult-1	no								
Mult-2	no								
Add	no								
Divide	no								

	F0	F2	F4	F6	F8	F10	...	F30
FU								

- 62 cycles necessary (in-order issue; out-of-order execution & commit)

Tomasulo's Algorithm vs. Scoreboard: the Example Results

Instruction	Issue	Execute	Write Result
L.D F6, 34(R2)	1	2	3
L.D F2, 45(R3)	2	3	4
MUL.D F0, F2, F4	3	14	15
SUB.D F8, F6, F2	4	6	7
DIV.D F10, F0, F6	5	55	56
ADD.D F6, F8, F2	6	9	10

- Tomasulo's algorithm did better because we had:
  - less structural hazards (two parallel loads)
  - CDB-based bypassing (division starts 1 cycle after multiplication)
  - no stalling for WAR hazards (addition starts earlier)

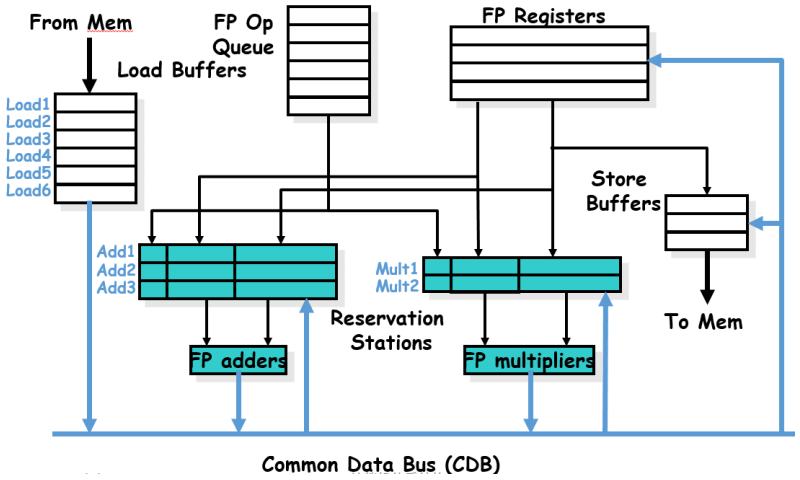
Instruction	Issue	Read Operands	Exec. Complete	Write Result
L.D F6, 34(R2)	1	2	3	4
L.D F2, 45(R3)	5	6	7	8
MUL.D F0, F2, F4	6	9	19	20
SUB.D F8, F6, F2	7	9	11	12
DIV.D F10, F0, F6	8	21	61	62
ADD.D F6, F8, F2	13	14	16	22

Tomasulo's Algorithm: Checking & Bookkeeping per Instruction (op D,S1,S2)

Inst. Status	Wait until	Bookkeeping
1. Issue	(RS[r].busy == no) where r is the reservation station for the functional unit matching op	if (Regs[S1].Qi ≠0) RS.[r].Qj ← Regs[S1].Qi else RS.[r].Vj ← Regs[S1].value if (Regs[S2].Qi ≠0) RS.[r].Qk ← Regs[S2].Qi else RS.[r].Vk ← Regs[S2].value RS[r].busy ← yes ; Regs[D].Qi ← r;
2. Execution	(RS.[r].Qj == 0) and (RS.[r].Qk == 0)	execute operation using operands in RS[r].Vj and RS[r].Vk and producing res
3. Write Result	RS.[r] is done & CDB is available	∀x (if (Regs[x].Qi == r) Regs[x].val ← res; Regs[x].Qi ← 0) ∀f (if (RS[f].Qj == r) RS[f].Vj ← res; RS[f].Qj ← 0) ∀f (if (RS[f].Qk == r) RS[f].Vk ← res; RS[f].Qk ← 0) RS[r].busy ← no

Tomasulo's Algorithm: Checking & Execution of Memory Accesses--Issue

Load: rs,基地址寄存器 ； rt : 目标寄存器存器；
Store: rs,基地址寄存器 ； rt : 源寄存器存器；
RS: 保留站数据结构；r:保留站编号
RegS：寄存器状态表
Reg： 寄存器堆



Load or Store:

Wait until: Buffer **r** empty

Action or bookkeeping:

if(RegS [rs].Qi≠0) ；基址寄存器未就绪
{RS[r].Qj ← RegS [rs].Qi}
else {RS[r].Vj ←Reg[rs]; RS[r].Qj ←0 } ； 等待基址寄存器生成
RS[r].A ←imm; RS[r].Busy ← yes; ； 获得指令中的偏移量

Load only: RegS[rt].Qi = r; (寄存器重命名) ； 目标寄存器数据源设置为保留站r

Store only:

if(RegS[rt].Qi≠0) {RS[r].Qk ← RegS[rt].Qi} ； 回写数据源设为生成rt的保留站
else {RS[r].Vk ←Reg[rt]; RS[r].Qk←0 } ；否则直接读取回写数据

Tomasulo's Algorithm: Execution of Memory Accesses

Loads and Stores executed *in program order* based on a two-step process

1. compute the effective address when base register is available
2. then
   - if load execute as soon as the memory unit is available
   - if store, wait for the value to be stored and send it to memory unit

Inst. Status	Wait until	Bookkeeping
2. Execution	(RS.[r].Qj == 0) and r being head of load/store queue	RS[r].A ← RS[r].Vj + RS[r].A if (op is load) do res ← Mem[RS[r].A]
3. Write Result	RS.[r] is done & CDB is available if (op is store) RS.[r] is done & wait for RS[r].Qk=0	if (op is load) same steps as before... if (op is store) do Mem[RS[r].A] ← RS[r].Vk RS[r].busy ← no

Tomasulo's Algorithm vs. Scoreboard

	Tomasulo's Algorithm	Scoreboard
introduced with	IBM 360/91 in 1967	CDC 6600 in 1964
resources	3 adders, 2 mul/div, 6 load, 3 store	7 int units, 4 FP units, 5 mem. reference
max window size	14	5
structural hazards	stall issuing	stall issuing
WAR hazards	avoided by register renaming	stall committing
WAW hazards	avoided by register renaming	stall issuing
control	based on reservation stations	scoreboard module
communication	broadcasting to reservation stations (and registers)	buses to/from registers

- With scoreboard both operands are read at once when they are both ready in the register file, while Tomasulo's algorithm reads each operand asap

- HW3:
  - 3.11
  - 3.12 (a) (b)

Tomasulo's Algorithm: Loop Example

```
loop: L.D F0, 0(R1)
      MUL.D F4, F0, F2
      S.D F4, 0(R1)
      DADDIU R1, R1, #-8
      BNEZ R1 loop
```

- Assumptions
  - R1 is initialized to #80
  - multiplies can complete in 4 clock cycles
  - first L.D takes 8 clock cycles (cache miss), following loads take 1 cycle
  - DADDIU takes one cycle
  - branch is predicted as taken

Tomasulo's Alg. (Loop Ex.): Clock Cycle = 1

Instruction	Issue	Execute	Write Result
L.D F0, 0(R1)	1		
MUL.D F4, F0, F2			
S.D F4, 0(R1)			
L.D F0, 0(R1)			
MUL.D F4, F0, F2			
S.D F4, 0(R1)			

- issuing first load

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	yes	L.D		80			0
Load-2	no						
Add-1	no						
Store-1	no						
Store-2	no						
Mult-1	no						
Mult-2	no						

Field	F0	F2	F4	F6	F8	F10	...	F30
Value								
Qi	Load-1							

Tomasulo's Alg. (Loop Ex.): Clock Cycle = 2

Instruction	Issue	Execute	Write Result
L.D F0, 0(R1)	1		
MUL.D F4, F0, F2	2		
S.D F4, 0(R1)			
L.D F0, 0(R1)			
MUL.D F4, F0, F2			
S.D F4, 0(R1)			

- executing first load (computing address but having cache miss)
- issuing multiplication

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	yes	L.D		80			80=0+80
Load-2	no						
Add-1	no						
Store-1	no						
Store-2	no						
Mult-1	yes	MUL.D		R[F2]	Load-1		
Mult-2	no						

Field	F0	F2	F4	F6	F8	F10	...	F30
Value								
Qi	Load-1		Mult-1					

Tomasulo's Alg. (Loop Ex.): Clock Cycle = 3

Instruction	Issue	Execute	Write Result
L.D F0, 0(R1)	1		
MUL.D F4, F0, F2	2		
S.D F4, 0(R1)	3		
L.D F0, 0(R1)			
MUL.D F4, F0, F2			
S.D F4, 0(R1)			

- executing first load (still 6 cycles to go)
- stalling multiplication (wait for first load)
- issuing first store

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	yes	L.D		80			80
Load-2	no						
Add-1	no						
Store-1	yes	S.D		80	Mult-1		0
Store-2	no						
Mult-1	yes	MUL.D		R[F2]	Load-1		
Mult-2	no						

Field	F0	F2	F4	F6	F8	F10	...	F30
Value								
Qi	Load-1		Mult-1					

Tomasulo's Alg. (Loop Ex.): Clock Cycle = 4

Instruction	Issue	Execute	Write Result
L.D F0, 0(R1)	1		
MUL.D F4, F0, F2	2		
S.D F4, 0(R1)	3	4	
L.D F0, 0(R1)			
MUL.D F4, F0, F2			
S.D F4, 0(R1)			

- issuing DADDIU (not shown)
- executing first load (still 5 cycles to go)
- stalling multiplication (wait for first load)
- first store has computed address, stalls in write result

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	yes	L.D		80			80
Load-2	no						
Add-1	no						
Store-1	yes	S.D		80	Mult-1		80=80+0
Store-2	no						
Mult-1	yes	MUL.D		R[F2]	Load-1		
Mult-2	no						

Field	F0	F2	F4	F6	F8	F10	...	F30
Value								
Qi	Load-1		Mult-1					

Tomasulo's Alg. (Loop Ex.): Clock Cycle = 5

Instruction	Issue	Execute	Write Result
L.D F0, 0(R1)	1		
MUL.D F4, F0, F2	2		
S.D F4, 0(R1)	3	4	
L.D F0, 0(R1)			
MUL.D F4, F0, F2			
S.D F4, 0(R1)			

- executing DADDIU (not shown)
- issuing BNEZ (not shown)
- executing first load (- 4)
- stalling multiplication
- stalling first store

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	yes	L.D		80			80
Load-2	no						
Add-1	no						
Store-1	yes	S.D		80	Mult-1		80
Store-2	no						
Mult-1	yes	MUL.D		R[F2]	Load-1		
Mult-2	no						

Field	F0	F2	F4	F6	F8	F10	...	F30
Value								
Qi	Load-1		Mult-1					

Tomasulo's Alg. (Loop Ex.): Clock Cycle = 6

Instruction	Issue	Execute	Write Result
L.D F0, 0(R1)	1		
MUL.D F4, F0, F2	2		
S.D F4, 0(R1)	3	4	
L.D F0, 0(R1)	6		
MUL.D F4, F0, F2			
S.D F4, 0(R1)			

- issuing second load (branch predicted correctly)
- F0 will never see the result of first load instruction!
- executing first load (- 3)
- stalling multiplication
- stalling first store

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	yes	L.D		80			80
Load-2	yes	L.D		72			0
Add-1	no						
Store-1	yes	S.D		80	Mult-1		80
Store-2	no						
Mult-1	yes	MUL.D		R[F2]	Load-1		
Mult-2	no						

Field	F0	F2	F4	F6	F8	F10	...	F30
Value								
Qi	Load-2		Mult-1					

Tomasulo's Alg. (Loop Ex.): Clock Cycle = 7

Instruction	Issue	Execute	Write Result
L.D F0, 0(R1)	1		
MUL.D F4, F0, F2	2		
S.D F4, 0(R1)	3	4	
L.D F0, 0(R1)	6		
MUL.D F4, F0, F2	7		
S.D F4, 0(R1)			

- issuing second multiplication (register file is by-passed)
- executing first load (-2)
- executing second load (computing address)
- stalling multiplication
- stalling first store

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	yes	L.D		80			80
Load-2	yes	L.D		72			72=72+0
Add-1	no						
Store-1	yes	S.D		80	Mult-1		80
Store-2	no						
Mult-1	yes	MUL.D		R[F2]	Load-1		
Mult-2	yes	MUL.D		R[F2]	Load-2		

Field	F0	F2	F4	F6	F8	F10	...	F30
Value								
Qi	Load-2		Mult-2					

Tomasulo's Alg. (Loop Ex.): Clock Cycle = 8

Instruction	Issue	Execute	Write Result
L.D F0, 0(R1)	1		
MUL.D F4, F0, F2	2		
S.D F4, 0(R1)	3	4	
L.D F0, 0(R1)	6		
MUL.D F4, F0, F2	7		
S.D F4, 0(R1)	8		

- issuing second store
- executing first load (-1)
- executing second load (-2)
- stalling multiplications
- stalling first store

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	yes	L.D		80			80
Load-2	yes	L.D		72			72
Add-1	no						
Store-1	yes	S.D		80	Mult-1		80
Store-2	yes	S.D		72	Mult-2		0
Mult-1	yes	MUL.D		R[F2]	Load-1		
Mult-2	yes	MUL.D		R[F2]	Load-2		

Field	F0	F2	F4	F6	F8	F10	...	F30
Value								
Qi	Load-2		Mult-2					

Tomasulo's Alg. (Loop Ex.): Clock Cycle = 9

Instruction	Issue	Execute	Write Result
L.D F0, 0(R1)	1	9	
MUL.D F4, F0, F2	2		
S.D F4, 0(R1)	3	4	
L.D F0, 0(R1)	6		
MUL.D F4, F0, F2	7		
S.D F4, 0(R1)	8	9	

- issuing second DADDIU
- executed first load
- executing second load (-1)
- stalling multiplication
- stalling first store
- second store has computed address, stalls in write result

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	yes	L.D		80			80
Load-2	yes	L.D		72			72
Add-1	no						
Store-1	yes	S.D		80	Mult-1		80
Store-2	yes	S.D		72	Mult-2		72=72+0
Mult-1	yes	MUL.D		R[F2]	Load-1		
Mult-2	yes	MUL.D		R[F2]	Load-2		

Field	F0	F2	F4	F6	F8	F10	...	F30
Value								
Qi	Load-2		Mult-2					

Tomasulo's Alg. (Loop Ex.): Clock Cycle = 10

Instruction	Issue	Execute	Write Result
L.D F0, 0(R1)	1	9	10
MUL.D F4, F0, F2	2		
S.D F4, 0(R1)	3	4	
L.D F0, 0(R1)	6	10	
MUL.D F4, F0, F2	7		
S.D F4, 0(R1)	8	9	

- issuing second BNEZ
- "committed" and broadcasted first load
- executed second load
- executing first multip. (-4)
- stalling second multip.
- stalling stores

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	no						
Load-2	yes	L.D		72			72
Add-1	no						
Store-1	yes	S.D		80	Mult-1		80
Store-2	yes	S.D		72	Mult-2		72
Mult-1	yes	MUL.D	Mem[80]	R[F2]			
Mult-2	yes	MUL.D		R[F2]	Load-2		

Field	F0	F2	F4	F6	F8	F10	...	F30
Value								
Qi	Load-2		Mult-2					

Tomasulo's Alg. (Loop Ex.): Clock Cycle = 11

Instruction	Issue	Execute	Write Result
L.D F0, 0(R1)	1	9	10
MUL.D F4, F0, F2	2	starting from 11	
S.D F4, 0(R1)	3	4	
L.D F0, 0(R1)	6	10	11
MUL.D F4, F0, F2	7		
S.D F4, 0(R1)	8	9	

- issuing third load
- "committed" and broadcasted second load
- executing first mult. (-3)
- executing second mult. (-4)
- stalling stores

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	yes	L.D		64			0
Load-2	no						
Add-1	no						
Store-1	yes	S.D		80	Mult-1		80
Store-2	yes	S.D		72	Mult-2		72
Mult-1	yes	MUL.D	Mem[80]	R[F2]			
Mult-2	yes	MUL.D	Mem[72]	R[F2]			

Field	F0	F2	F4	F6	F8	F10	...	F30
Value								
Qi	Load-1		Mult-2					

Tomasulo's Alg. (Loop Ex.): Clock Cycle = 12

Instruction	Issue	Execute	Write Result
L.D F0, 0(R1)	1	9	10
MUL.D F4, F0, F2	2	starting from 11	
S.D F4, 0(R1)	3	4	
L.D F0, 0(R1)	6	10	11
MUL.D F4, F0, F2	7	starting from 12	
S.D F4, 0(R1)	8	9	

- stall issuing of third multiplication (due to structural hazard)
- executing first mult. (-2)
- executing second mult. (-3)
- stalling stores

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	yes	L.D		64			64=64+0
Load-2	no						
Add-1	no						
Store-1	yes	S.D		80	Mult-1		80
Store-2	yes	S.D		72	Mult-2		72
Mult-1	yes	MUL.D	Mem[80]	R[F2]			
Mult-2	yes	MUL.D	Mem[72]	R[F2]			

Field	F0	F2	F4	F6	F8	F10	...	F30
Value								
Qi	Load-1		Mult-2					

Tomasulo's Alg. (Loop Ex.): Clock Cycle = 13

Instruction	Issue	Execute	Write Result
L.D F0, 0(R1)	1	9	10
MUL.D F4, F0, F2	2	starting from 11	
S.D F4, 0(R1)	3	4	
L.D F0, 0(R1)	6	10	11
MUL.D F4, F0, F2	7	starting from 12	
S.D F4, 0(R1)	8	9	

- issuing stage is still stalled
- executing first mult. (-1)
- executing second mult. (-2)
- stalling stores

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	yes	L.D		64			64
Load-2	no						
Add-1	no						
Store-1	yes	S.D		80	Mult-1		80
Store-2	yes	S.D		72	Mult-2		72
Mult-1	yes	MUL.D	Mem[80]	R[F2]			
Mult-2	yes	MUL.D	Mem[72]	R[F2]			

Field	F0	F2	F4	F6	F8	F10	...	F30
Value								
Qi	Load-1		Mult-2					

Tomasulo's Alg. (Loop Ex.): Clock Cycle = 14

Instruction	Issue	Execute	Write Result
L.D F0, 0(R1)	1	9	10
MUL.D F4, F0, F2	2	14	
S.D F4, 0(R1)	3	4	
L.D F0, 0(R1)	6	10	11
MUL.D F4, F0, F2	7	starting from 12	
S.D F4, 0(R1)	8	9	

- issuing stage is still stalled
- executed first multiplication
- executing second mult. (-1)
- stalling stores

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	yes	L.D		64			64
Load-2	no						
Add-1	no						
Store-1	yes	S.D		80	Mult-1		80
Store-2	yes	S.D		72	Mult-2		72
Mult-1	yes	MUL.D	Mem[80]	R[F2]			
Mult-2	yes	MUL.D	Mem[72]	R[F2]			

Field	F0	F2	F4	F6	F8	F10	...	F30
Value								
Qi	Load-1		Mult-2					

Tomasulo's Alg. (Loop Ex.): Clock Cycle = 15

Instruction	Issue	Execute	Write Result
L.D F0, 0(R1)	1	9	10
MUL.D F4, F0, F2	2	14	15
S.D F4, 0(R1)	3	4	
L.D F0, 0(R1)	6	10	11
MUL.D F4, F0, F2	7	15	
S.D F4, 0(R1)	8	9	

- issuing stage is still stalled
- "committing" & broadcasting first multiplication
- executed second multiplication
- stalling stores

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	yes	L.D		64			64
Load-2	no						
Add-1	no						
Store-1	yes	S.D	M[80]*R[F2]	80			80
Store-2	yes	S.D		72	Mult-2		72
Mult-1	no						
Mult-2	yes	MUL.D	Mem[72]	R[F2]			

Field	F0	F2	F4	F6	F8	F10	...	F30
Value								
Qi	Load-1		Mult-2					

Tomasulo's Alg. (Loop Ex.): Clock Cycle = 16

Instruction	Issue	Execute	Write Result
L.D F0, 0(R1)	1	9	10
MUL.D F4, F0, F2	2	14	15
S.D F4, 0(R1)	3	4	16
L.D F0, 0(R1)	6	10	11
MUL.D F4, F0, F2	7	15	16
S.D F4, 0(R1)	8	9	

- issuing third multiplication (not shown on the left)
- "committing" & broadcasting second multiplication
- first store writes to memory
- stalling second store

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	yes	L.D					64
Load-2	no						
Add-1	no						
Store-1	yes	S.D	M[80] * R[F2]				80
Store-2	yes	S.D	M[72] * R[F2]				72
Mult-1	yes	MUL.D		R[F2]	Load-1		
Mult-2	no						

Field	F0	F2	F4	F6	F8	F10	...	F30
Value								
Qi	Load-1		Mult-1					

Tomasulo's Algorithm: Loop Example

Instruction	Issue	Execute	Write Result
L.D F0, 0(R1)	1	9	10
MUL.D F4, F0, F2	2	14	15
S.D F4, 0(R1)	3	4	16
L.D F0, 0(R1)	6	10	11
MUL.D F4, F0, F2	7	15	16
S.D F4, 0(R1)	8	9	

- Once the system reaches the steady state in the execution of this loop, two copies of the loop can be sustained with CPI close to 1.0
  - provided that there are enough resources and that the multiplication can complete in 4 cycles
- With multiple-instruction issue, Tomasulo's algorithm can sustain more than one instruction per clock!

Tomasulo's Algorithm: Dynamic Memory Disambiguation

- Load and stores accessing different addresses can be done out-of-order
- When accessing the same address

- with load before store, avoid WAR hazard
- with store before load, avoid RAW hazard
- with two stores, avoid WAW hazard

- Possible strategy

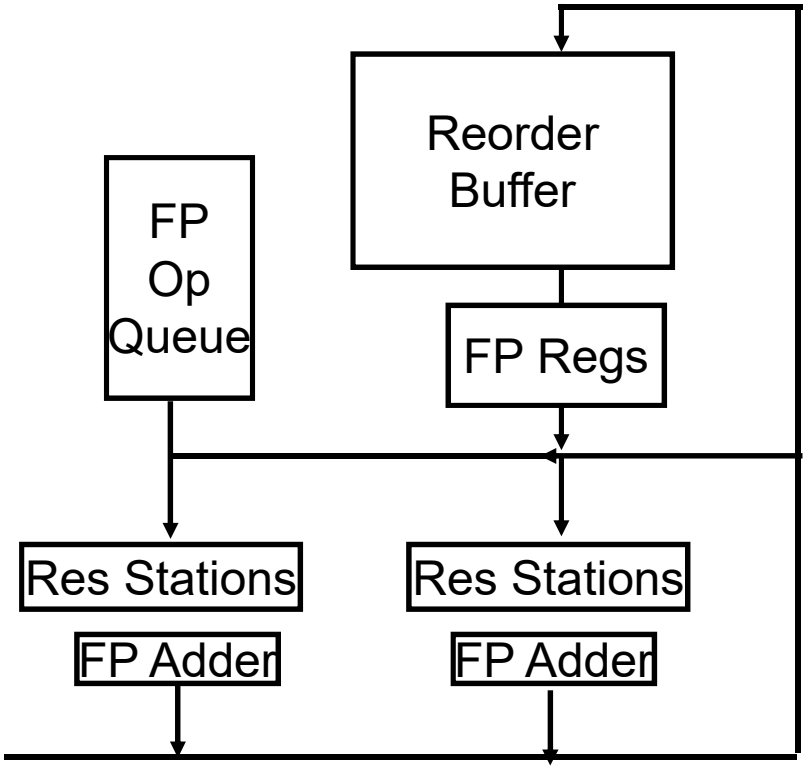
1. **in-order** effective address computation
2. for each load (store) check whether there is an **address conflict** by analyzing the A-field of all active store (load & store) buffers.
3. If there is a match, the new instruction is not sent to the load buffer until the old one completes

Tomasulo's Algorithm: Summary

- Main advantages
  - distribution of control & hazard detection logic
  - elimination of stalls for WAR and WAW hazards via register renaming
  - CDB-based by-passing mechanism via broadcasting from functional units to reservation stations
- Issues
  - implementation issues
    - design complexity (fast associative buffers)
    - CDB bottleneck (many CDB must interact with all reserv. stations)
  - interrupt model is not precise (due to out-of-order committing)
    - in the first example, SUB.D and ADD.D could both complete before MUL.D, which raises an exception
      - hardware support for speculation will solve the problem

硬件支持精确中断

- 需要硬件缓存没有提交的指令结果: reorder buffer (ROB)
  - 3 个域: 指令类型,目的地址, 值
  - Reorder buffer 可以作为操作数源 => 就像有更多的寄存器（与RS类似）
  - 当指令执行阶段完成后，用ROB的编号代替RS中的值
  - 增加指令提交阶段
  - ROB提供执行完成阶段和提交阶段的操作数
  - 一旦操作数提交，结果就写入寄存器
  - 在预测失败时，容易恢复推断执行的指令，或发生异常时，容易恢复状态



支持推断执行的 Tomasulo 算法的四阶段

1. Issue—get instruction from FP Op Queue
   - 如果RS和ROB有空闲单元就发射指令。如果寄存器或ROB中源操作数可用，就将其发送到RS，目的地址的ROB编号也发送给RS
2. Execution—operate on operands (EX)
   - 当操作数就绪后，开始执行。如果没有就绪，监测CDB，检查RAW相关
3. Write result—finish execution (WB)
   - 将运算结果通过CDB传送给所有等待结果的FU以及ROB单元，标识RS可用
4. Commit—update register with reorder result
   - 如果指令没有不确定性就可以提交
   - 按ROB表中顺序，如果结果已有，就更新寄存器（或存储器），并将该指令从ROB表中删除
   - 预测失败或有中断时，刷新ROB

例如：

```
L.D      F6, 34(R2)
L.D      F2, 45(R3)
MUL.D    F0, F2, F4
SUB.D    F8, F6, F2
DIV.D    F10, F0, F6
ADD.D    F6, F8, F2
```

假设：执行阶段的周期数:

LD： 1 cycles MULT: 9 cycles SUBD/ADDD: 2cycles DIVD: 40 cycles

Issue

op rd,rs,rt | L.D rt,(imm)rs | S.D rt,(imm)rs

Status	Wait until	Action or bookkeeping

Issue all instructions	Reservation station (r) and ROB (b) both available	`if (RegisterStat[rs].Busy)/*in-flight instr. writes rs*/`
		`{h ← RegisterStat[rs].Reorder;`
		`if (ROB[h].Ready)/* Instr completed already */`
		`{RS[r].Vj ← ROB[h].Value; RS[r].Qj ← 0;}`
		`else {RS[r].Qj ← h;} /* wait for instruction */`
		`} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0;};`
		`RS[r].Busy ← yes; RS[r].Dest ← b;`
		`ROB[b].Instruction ← opcode; ROB[b].Dest ← rd;ROB[b].Ready ← no;`

FP operations and stores		`if (RegisterStat[rt].Busy) /*in-flight instr writes rt*/`
		`{h ← RegisterStat[rt].Reorder;`
		`if (ROB[h].Ready)/* Instr completed already */`
		`{RS[r].Vk ← ROB[h].Value; RS[r].Qk ← 0;}`
		`else {RS[r].Qk ← h;} /* wait for instruction */`
		`} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0;};`

FP operations		`RegisterStat[rd].Reorder ← b; RegisterStat[rd].Busy ← yes;` `ROB[b].Dest ← rd;`
Loads		`RS[r].A ← imm; RegisterStat[rt].Reorder ← b;` `RegisterStat[rt].Busy ← yes; ROB[b].Dest ← rt;`
Stores		`RS[r].A ← imm;`



RegisterStat

Execute

op rd,rs,rt | L.D rt,(imm)rs | S.D rt,(imm)rs

	Wait until	Operation & Book keeping
Execute FP op	(RS[r].Qj == 0) and (RS[r].Qk == 0)	Compute results—operands are in Vj and Vk
Load step 1	(RS[r].Qj == 0) and there are no stores earlier in the queue	`RS[r].A ← RS[r].Vj + RS[r].A;`
Load step 2	Load step 1 done and all stores earlier in ROB have different address	Read from Mem[RS[r].A]
Store	(RS[r].Qj == 0) and store at queue head	`ROB[h].Address ← RS[r].Vj + RS[r].A;`



RegisterStat

Write result & Commit

Wait until

Operation & Book keeping

Write result all but store	Execution done at r and CDB available	b ← RS[r].Dest; RS[r].Busy ← no; ∀x(if (RS[x].Qj==b) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk==b) {RS[x].Vk ← result; RS[x].Qk ← 0}); ROB[b].Value ← result; ROB[b].Ready ← yes;
Store	Execution done at r and (RS[r].Qk == 0)	ROB[h].Value ← RS[r].Vk;
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;};} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;};

Op: Operation to perform in the unit (e.g., + or –)

Vj, Vk: Value of Source operands

- Store buffers has V field, result to be stored

Qj, Qk: Reservation stations producing source registers (value to be written)

- Note: No ready flags as in Scoreboard; Qj,Qk=0 => ready
- Store buffers only have Qi for RS producing result

A: to hold memory address for load and store instructoins

Busy: Indicates reservation station or FU is busy

Tomasulo With Reorder Buffer - Cycle 0

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Dest
0	Add1	No						
0	Add2	No						
0	Add3	No						
0	Mult1	No						
0	Mult2	No						

Reservation Stations

	Busy	Address
Load1		
Load2		
Load3		

Entry	Busy	Instruction	State	Destination	Value
1					
2					
3					
4					
5					
6					
7					
8					
9					
10					

Reorder Buffer

	F0	F2	F4	F6	F8	F10	F12	...	F30
Reorder #									
Busy	no	no	no	no	no	no	no		no

Tomasulo With Reorder Buffer - Cycle 1

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Dest
0	Add1	No						
0	Add2	No						
0	Add3	No						
0	Mult1	No						
0	Mult2	No						

Reservation Stations

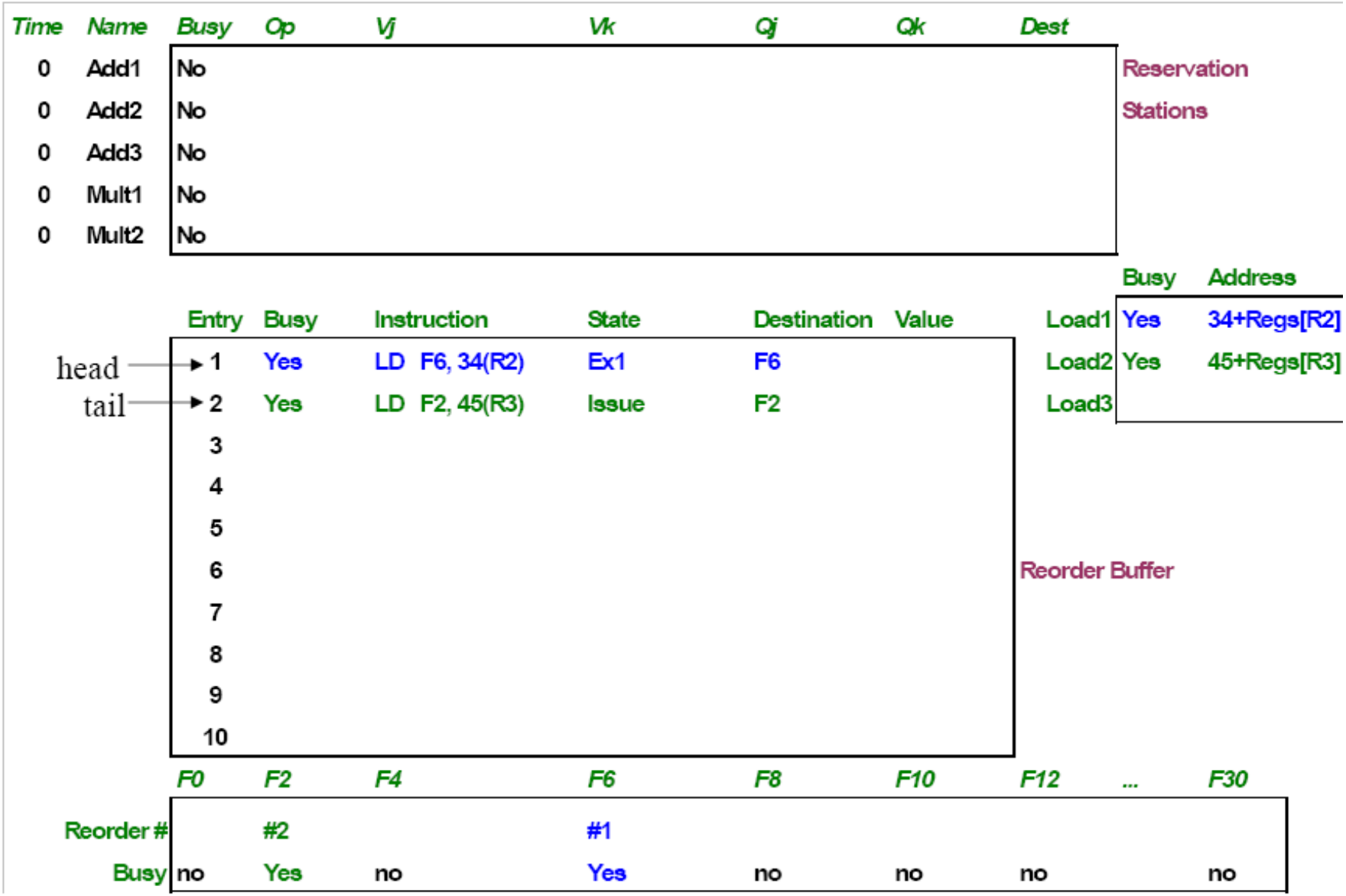
	Busy	Address
Load1	Yes	34+Regs[R2]
Load2		
Load3		

Entry	Busy	Instruction	State	Destination	Value
1	Yes	LD F6, 34(R2)	Issue	F6	
2					
3					
4					
5					
6					
7					
8					
9					
10					

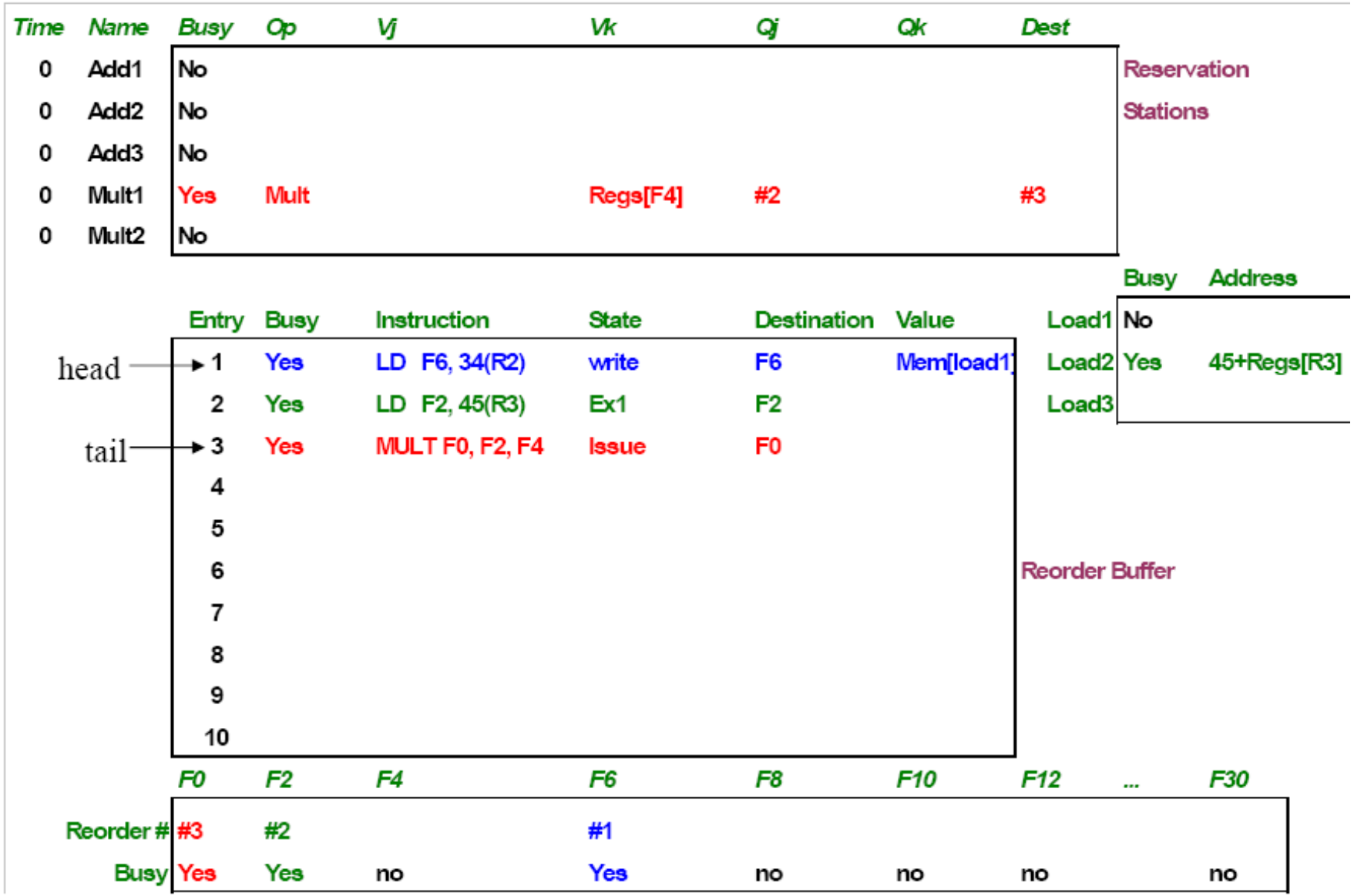
Reorder Buffer

	F0	F2	F4	F6	F8	F10	F12	...	F30
Reorder #				#1					
Busy	no	no	no	Yes	no	no	no		no

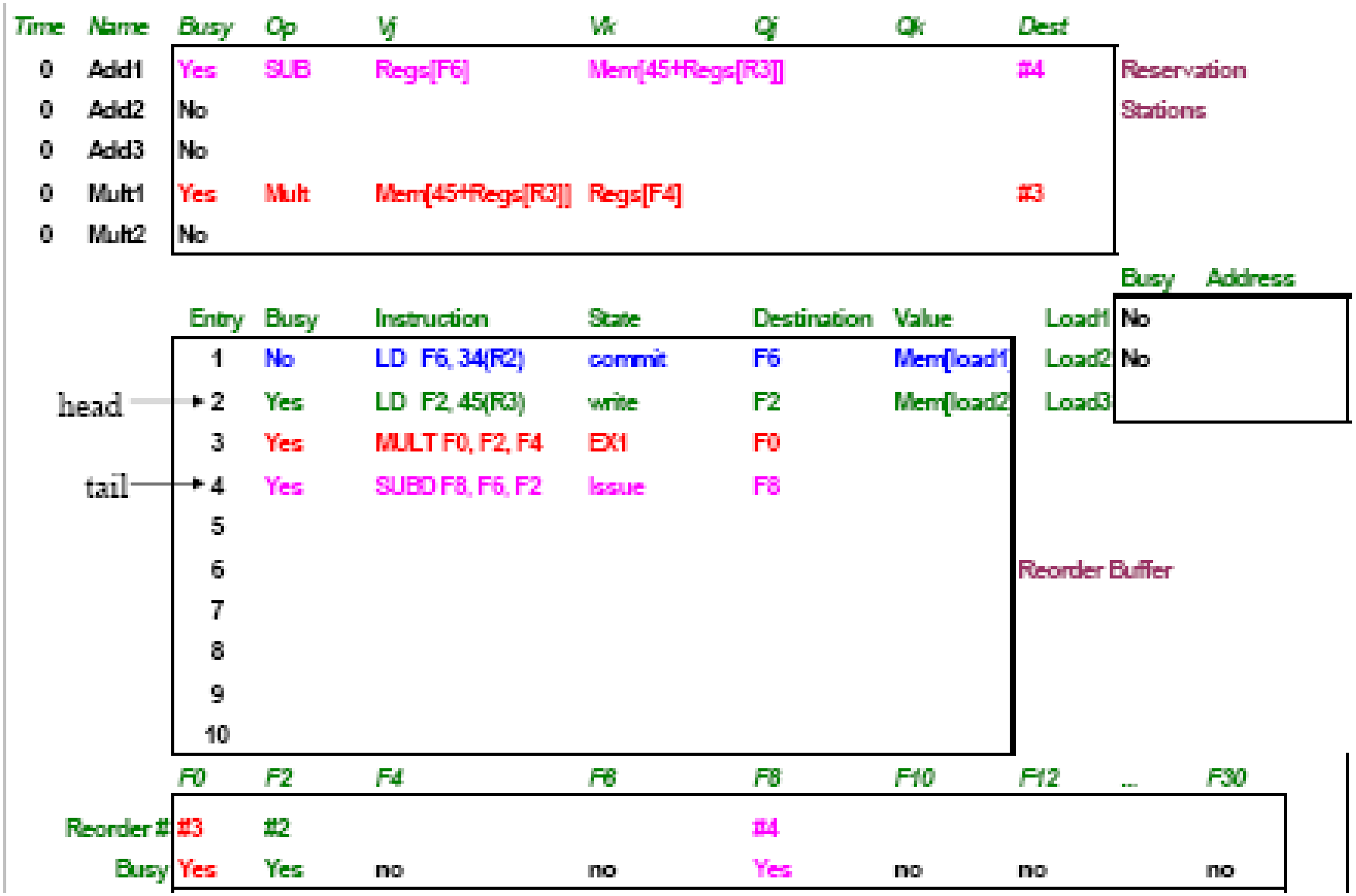
Tomasulo With Reorder Buffer - Cycle 2



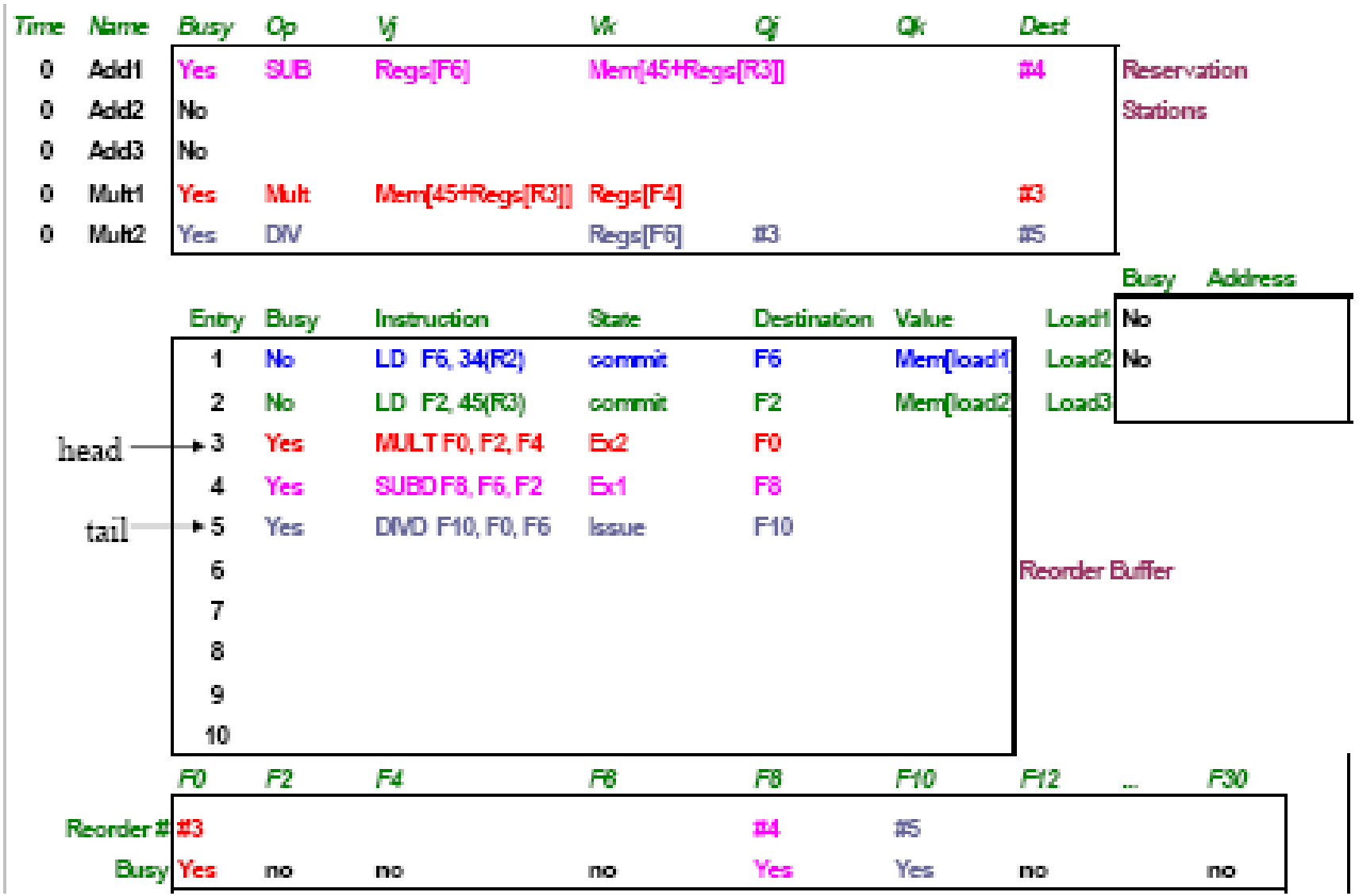
Tomasulo With Reorder Buffer - Cycle 3



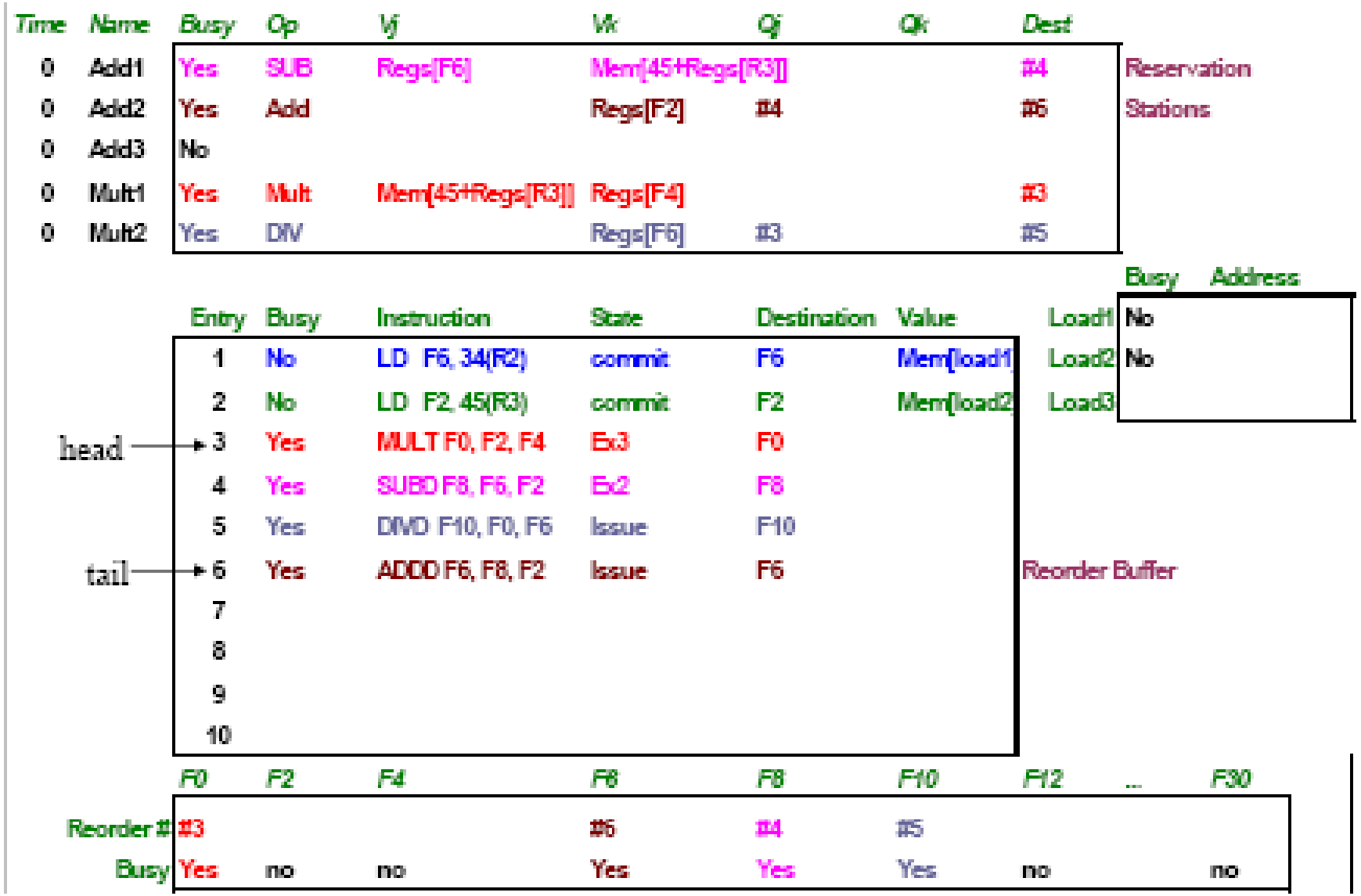
Tomasulo With Reorder Buffer - Cycle 4



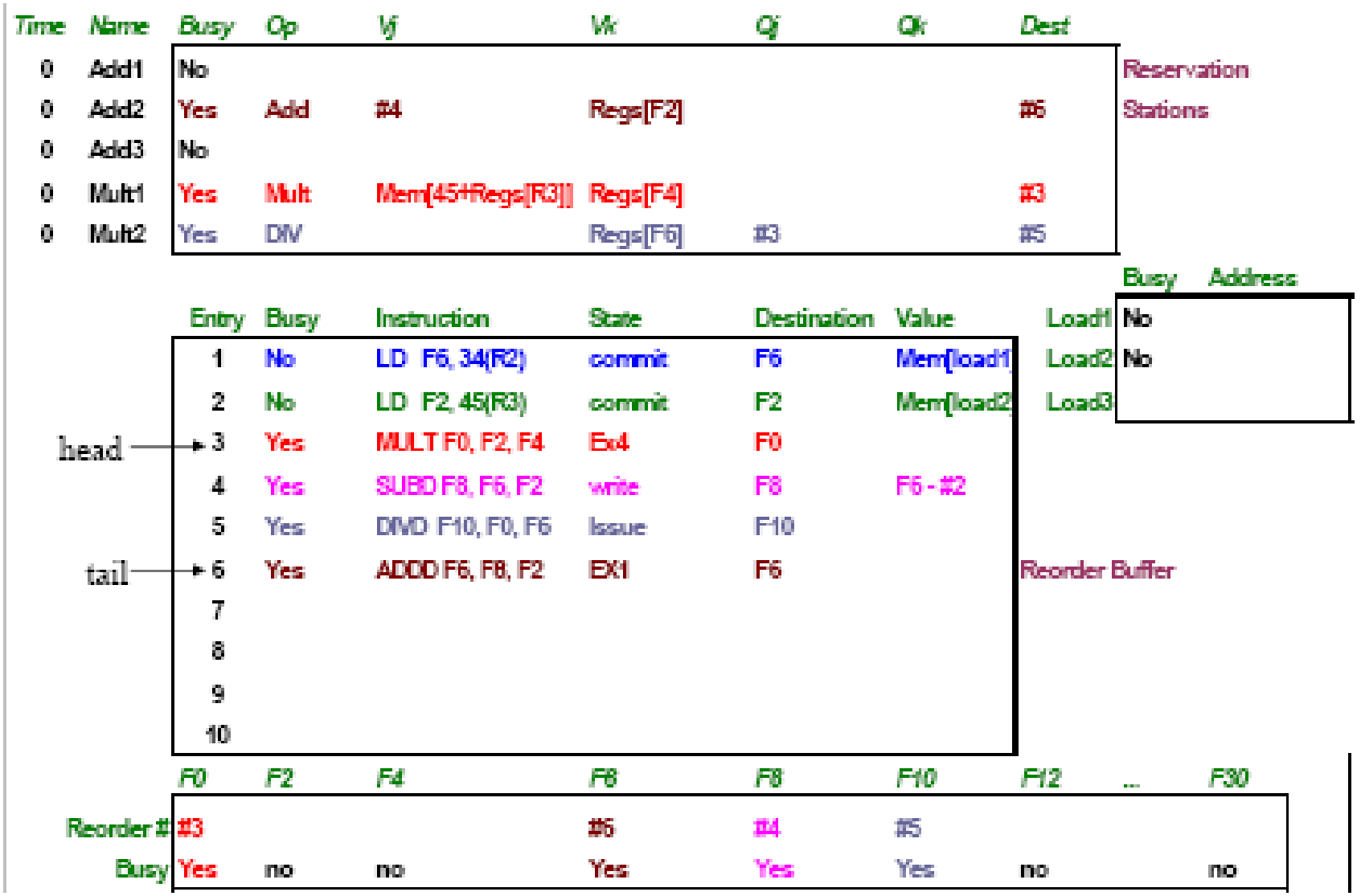
Tomasulo With Reorder Buffer - Cycle 5



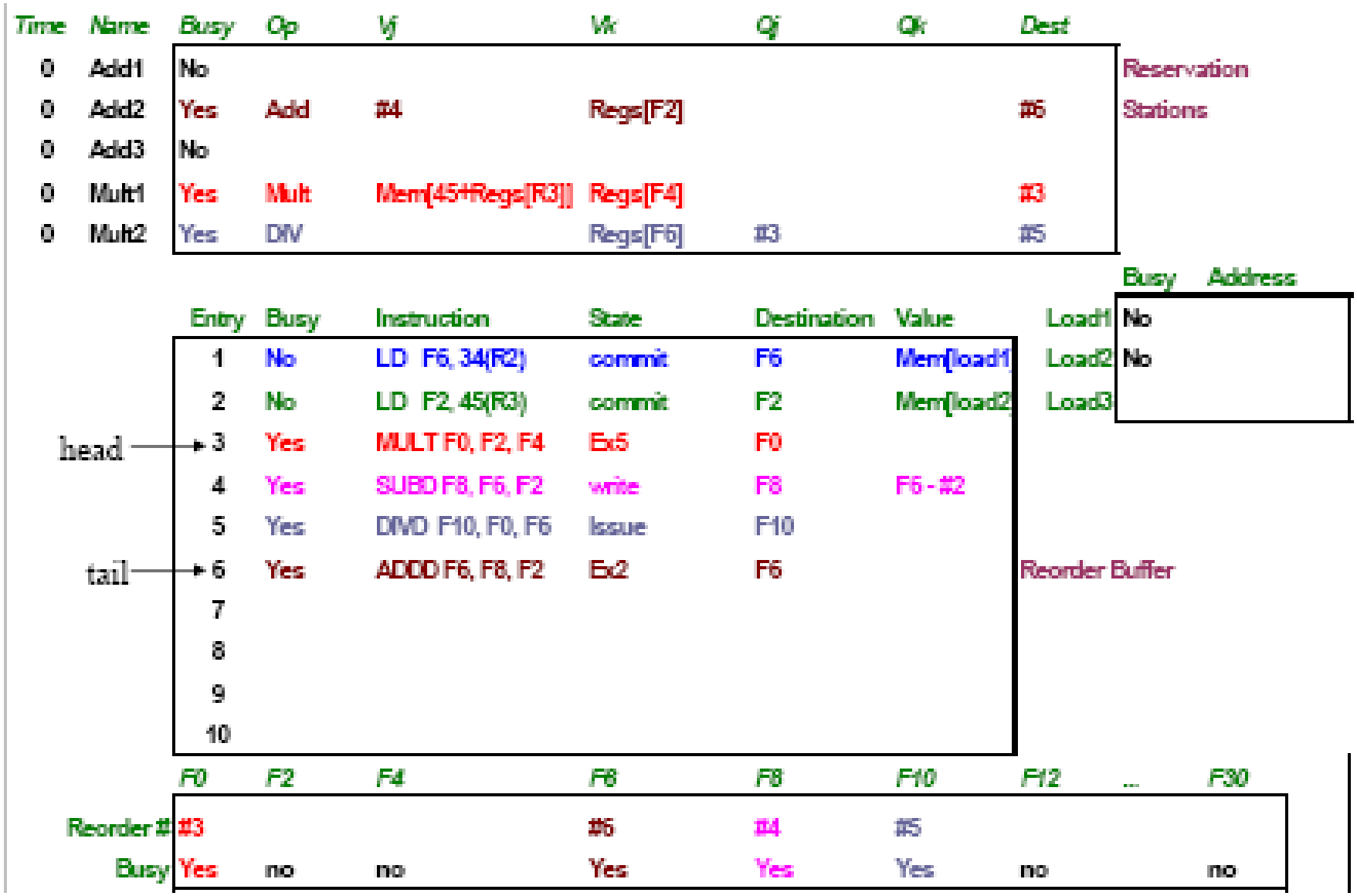
Tomasulo With Reorder Buffer - Cycle 6



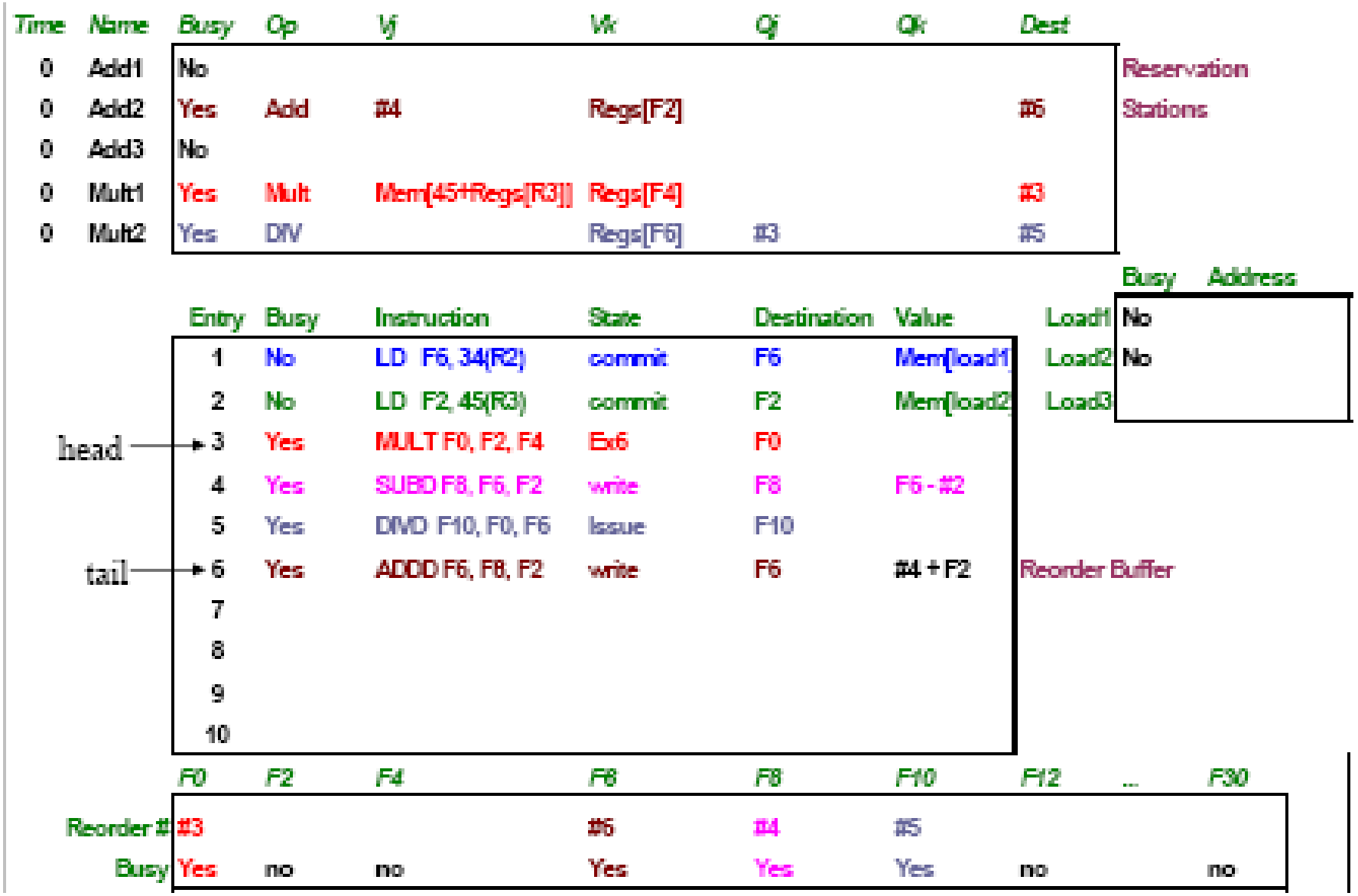
Tomasulo With Reorder Buffer - Cycle 7



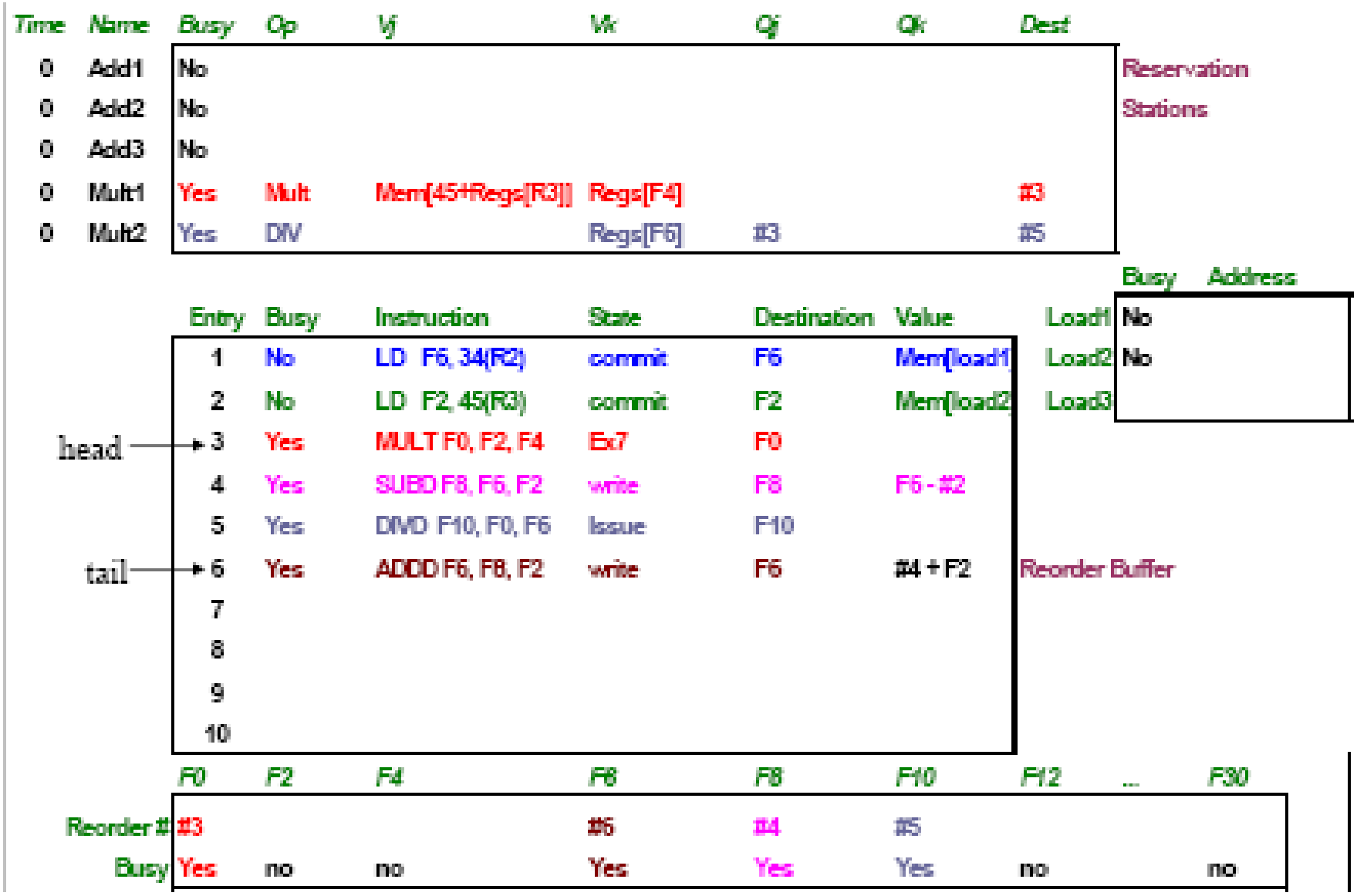
Tomasulo With Reorder Buffer - Cycle 8



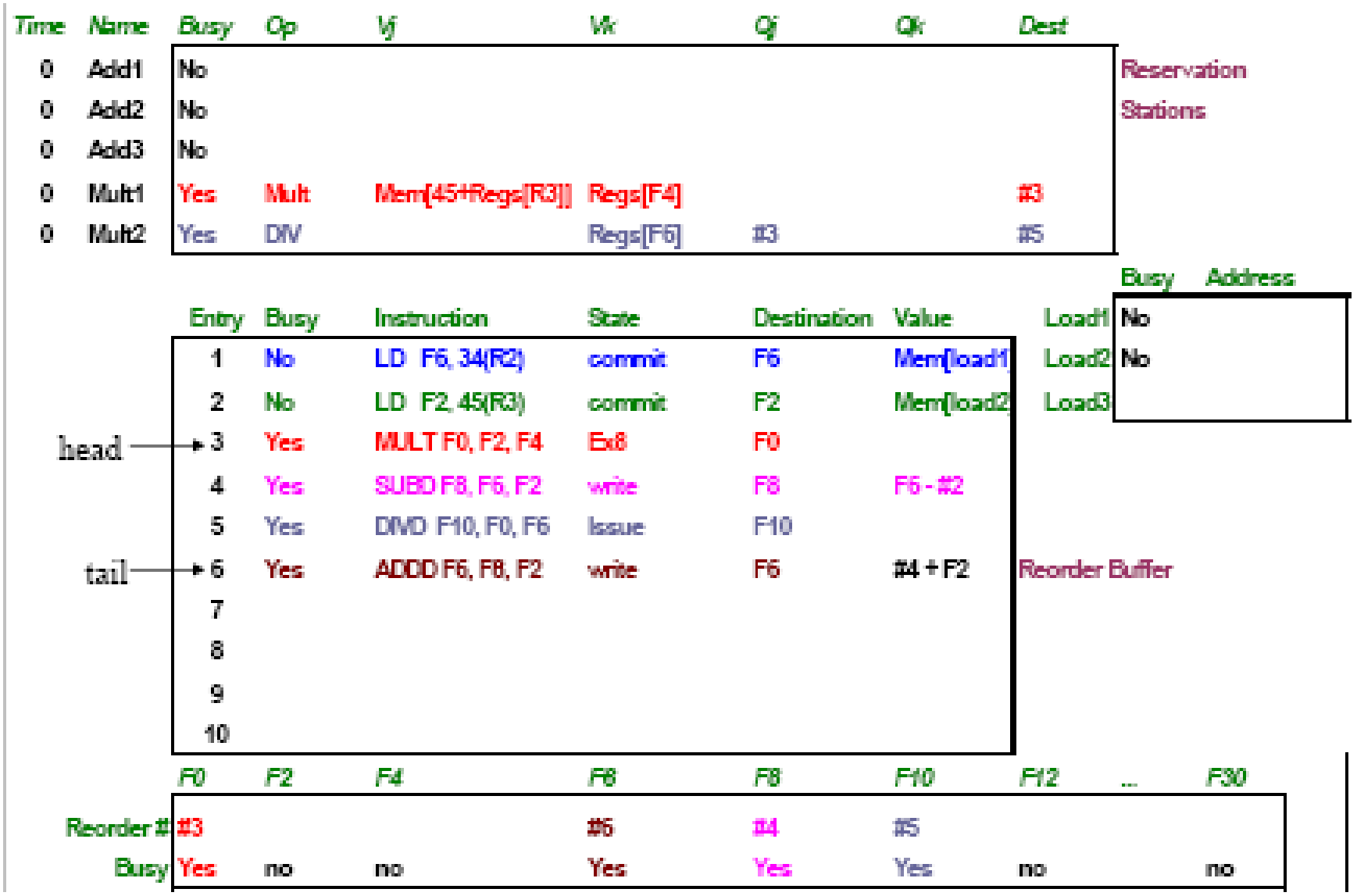
Tomasulo With Reorder Buffer - Cycle 9



Tomasulo With Reorder Buffer - Cycle 10



Tomasulo With Reorder Buffer - Cycle 11



Tomasulo With Reorder Buffer - Cycle 12

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Dest
0	Add1	No						
0	Add2	No						
0	Add3	No						
0	Mult1	Yes	Mult	Mem[45+Regs[R3]]	Regs[F4]			#3
0	Mult2	Yes	DIV		Regs[F6]	#3		#5

Reservation Stations

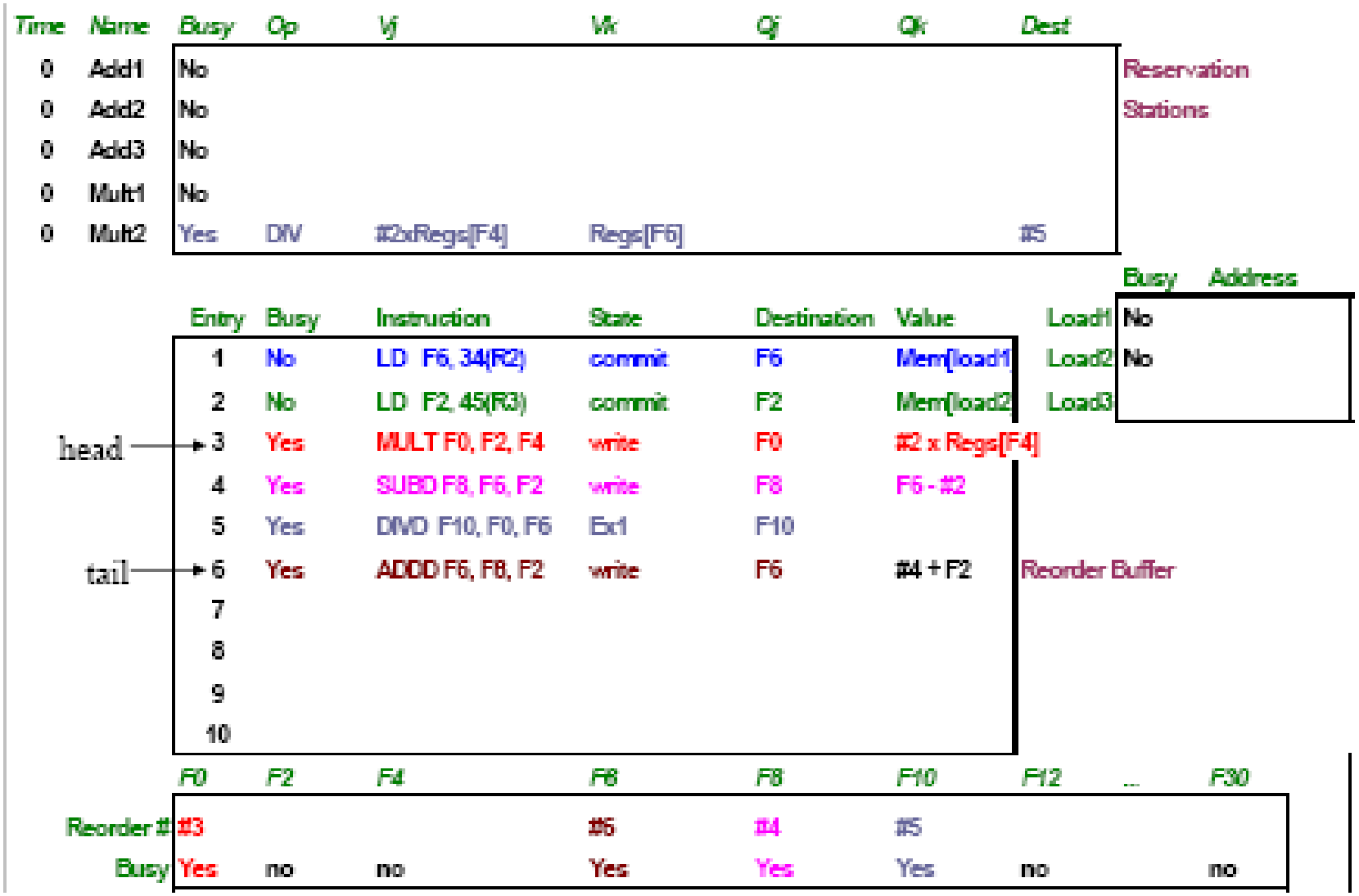
	Busy	Address
Load1	No	
Load2	No	
Load3		

	Entry	Busy	Instruction	State	Destination	Value
	1	No	LD F6, 34(R2)	commit	F6	Mem[load1]
	2	No	LD F2, 45(R3)	commit	F2	Mem[load2]
head →	3	Yes	MULT F0, F2, F4	Ex9	F0	
	4	Yes	SUBD F8, F6, F2	write	F8	F6 - #2
	5	Yes	DIVD F10, F0, F6	Issue	F10	
tail →	6	Yes	ADDD F6, F8, F2	write	F6	#4 + F2
	7					
	8					
	9					
	10					

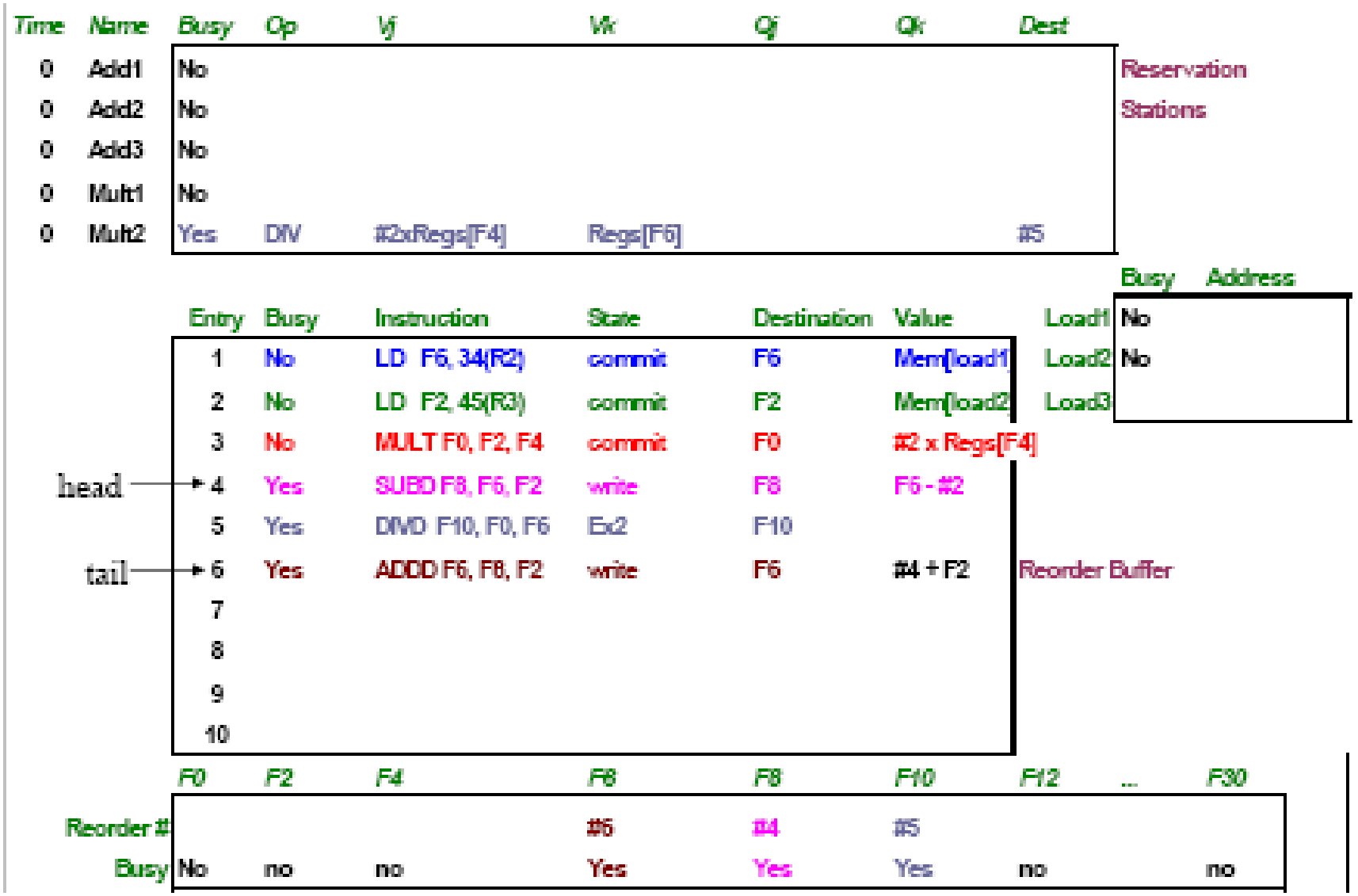
Reorder Buffer

	F0	F2	F4	F6	F8	F10	F12	...	F30
Reorder #	#3			#6	#4	#5			
Busy	Yes	no	no	Yes	Yes	Yes	no		no

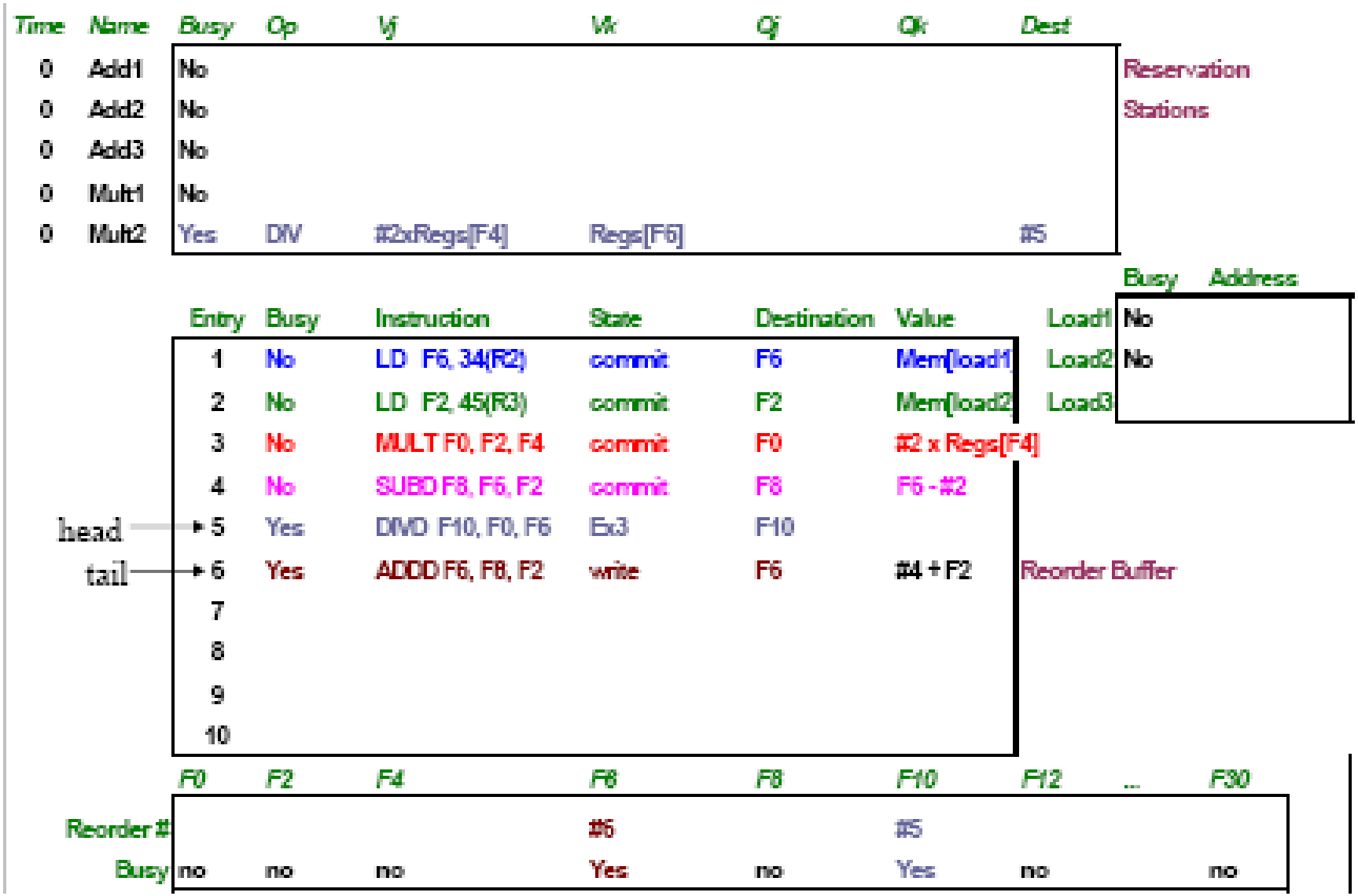
Tomasulo With Reorder Buffer - Cycle 13



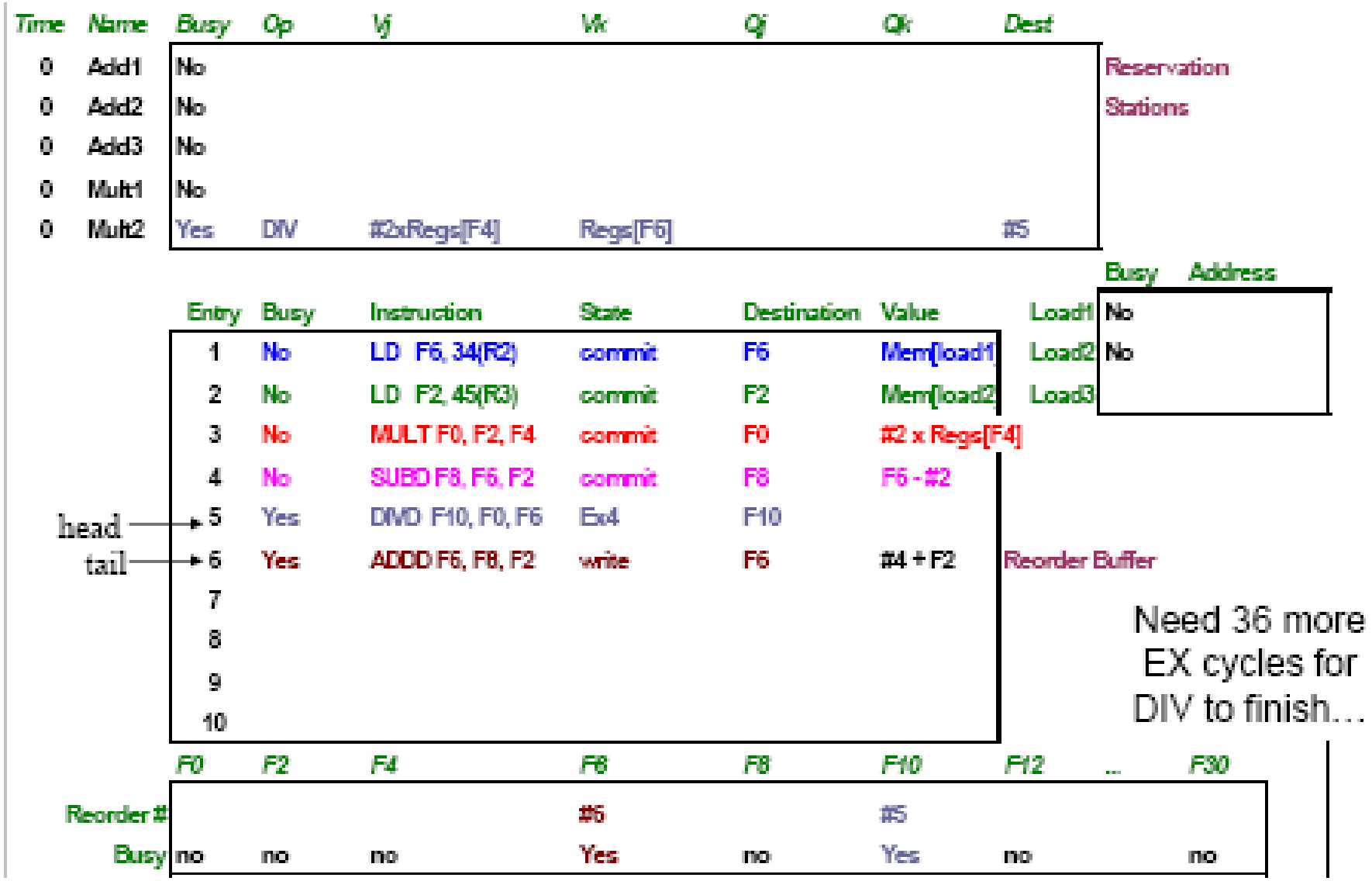
Tomasulo With Reorder Buffer - Cycle 14



Tomasulo With Reorder Buffer - Cycle 15



Tomasulo With Reorder Buffer - Cycle 16



Tomasulo With Reorder Buffer: Summary

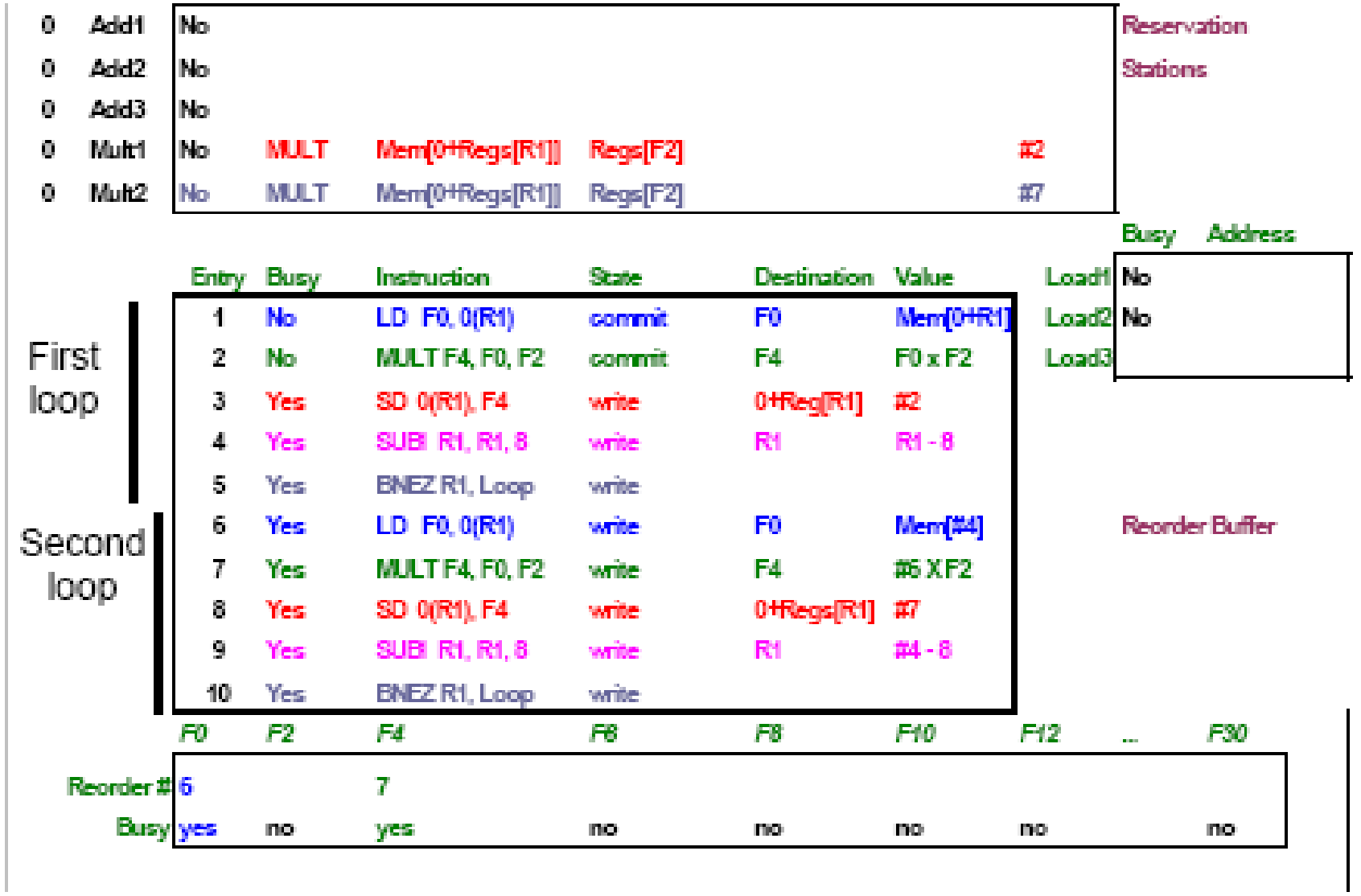
Instruction	Issue	Exec Comp	Writeback	Commit
LD F6, 34(R2)	1	2	3	4
LD F2, 45(R3)	2	3	4	5
MULT F0, F2, F4	3	12	13	14
SUBD F8, F6, F2	4	6	7	15
DIVD F10, F0, F6	5	52	53	54
ADDD F6, F8, F2	6	8	9	55

In-order Issue/Commit, Out-of-Order Execution/Writeback

Precise State with ROB

- ROB maintains precise state and allows speculation
  - Waits until precise condition reaches retire/commit stage
  - (Or until branch is noted mis-predicted)
  - Clear ROB, RS, and register status table (Flush)
  - Service exception/Restart from True Branch target
- Need to do similar things with memory ops
  - Called Memory Ordering Buffer (MOB)
    - Completed stores write to MOB then complete (write to memory) in-order (when they read head of buffer)

Example of Speculative State of Reorder Buffer



Multiply has just reached commit, so other instructions can start committing

Reorder Buffer -Review

- Reorder Buffer:
  - 提供了撤销指令运行的机制
  - 指令以发射序存放在ROB中
  - 指令顺序提交
- 分支预测对提高性能是非常重要的
  - 推断执行: 在控制相关还没有解决的情况下，开始执行
  - 推断执行是利用了ROB撤销指令执行的机制
- Memory Disambiguation
  - Nonspeculative Memory Disambiguation
  - Speculative Memory Disambiguation

消除存储器引用的歧义性：处理对存储器引用的RAW相关

- Question: 给定一个指令序列，store，load 这两个操作是否有关？
  - 即下列代码是否有相关问题?

```
Eg:     st      0(R2),R5
        ld      R6,0(R3)
```

- 我们是否可以较早启动ld?
  - Store的地址可能会延迟很长时间才能得到.
  - 我们也许想在同一个周期开始这两个操作的执行.
- 两种方法:
  - No Speculation: 不进行load操作，直到我们确信地址 0(R2) ≠ 0(R3)
  - Speculation: 我们可以假设他们相关还是不相关 (called "dependence speculation")，如果推测错误通过ROB来修正

Nonspeculative Memory Disambiguation

TABLE 6.1: Memory disambiguation schemes.

NAME	SPECULATIVE	DESCRIPTION
Total Ordering	No	All memory accesses are processed in order.
Partial Ordering	No	All stores are processed in order, but loads execute out of order as long as all previous stores have computed their address.
Load Ordering Store Ordering	No	Execution between loads and stores is out of order, but all loads execute in order among them, and all stores execute in order among them.
Store Ordering	Yes	Stores execute in order, but loads execute completely out of order.

非投机方式的基本原则：当前存储器指令之前的store指令计算存储器地址后，才能执行当前的Load存储器操作

Load Ordering. Store Ordering

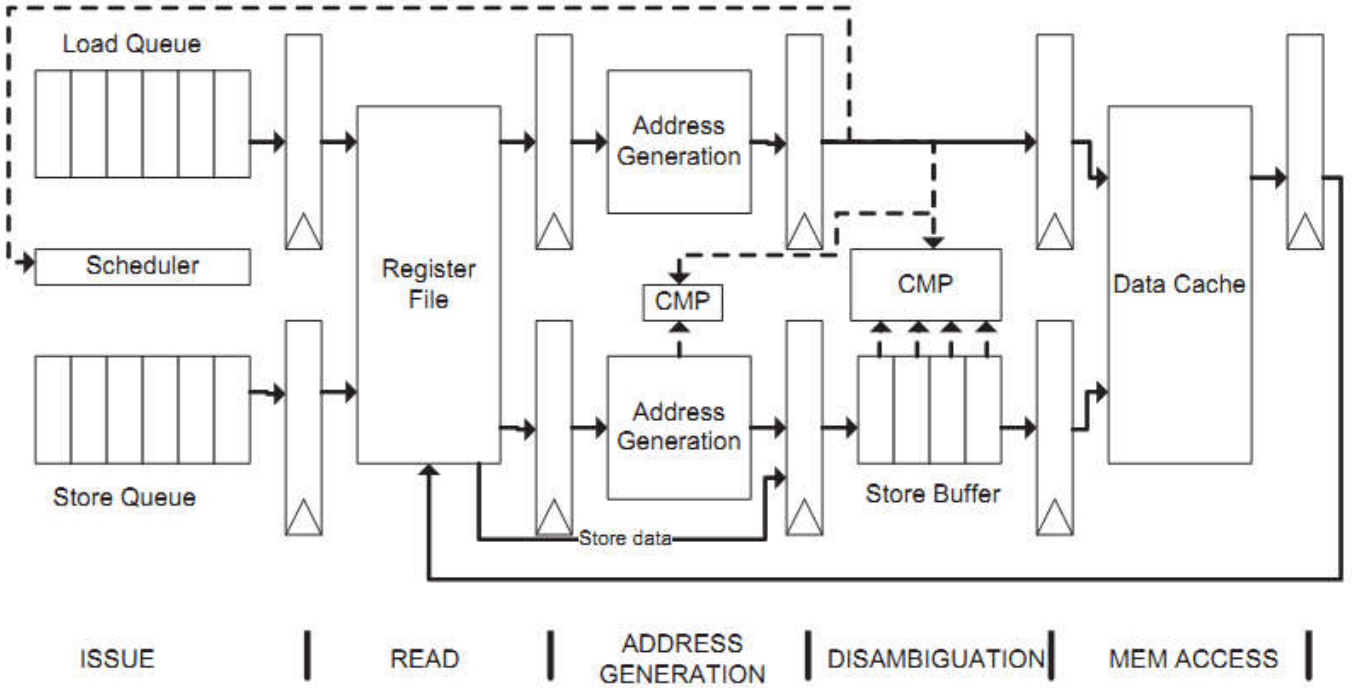


FIGURE 6.7: Schematics of the AMD K6 pipeline to implement a load-ordering store-ordering memory disambiguation policy.

Load queue: 以程序序存储load指令，按照FIFO方式流出

Address generation: 生成存储器访问有效地址

Store queue: 以程序序存储store指令，按照FIFO方式流出

Store buffer: 以程序序保留store操作（有效地址，值），一直到其成为最早的store操作，才实际更新存储器

CMP : 进行地址相关的比较的部件，用于地址消歧。

Partial Ordering （MIPS R10000）

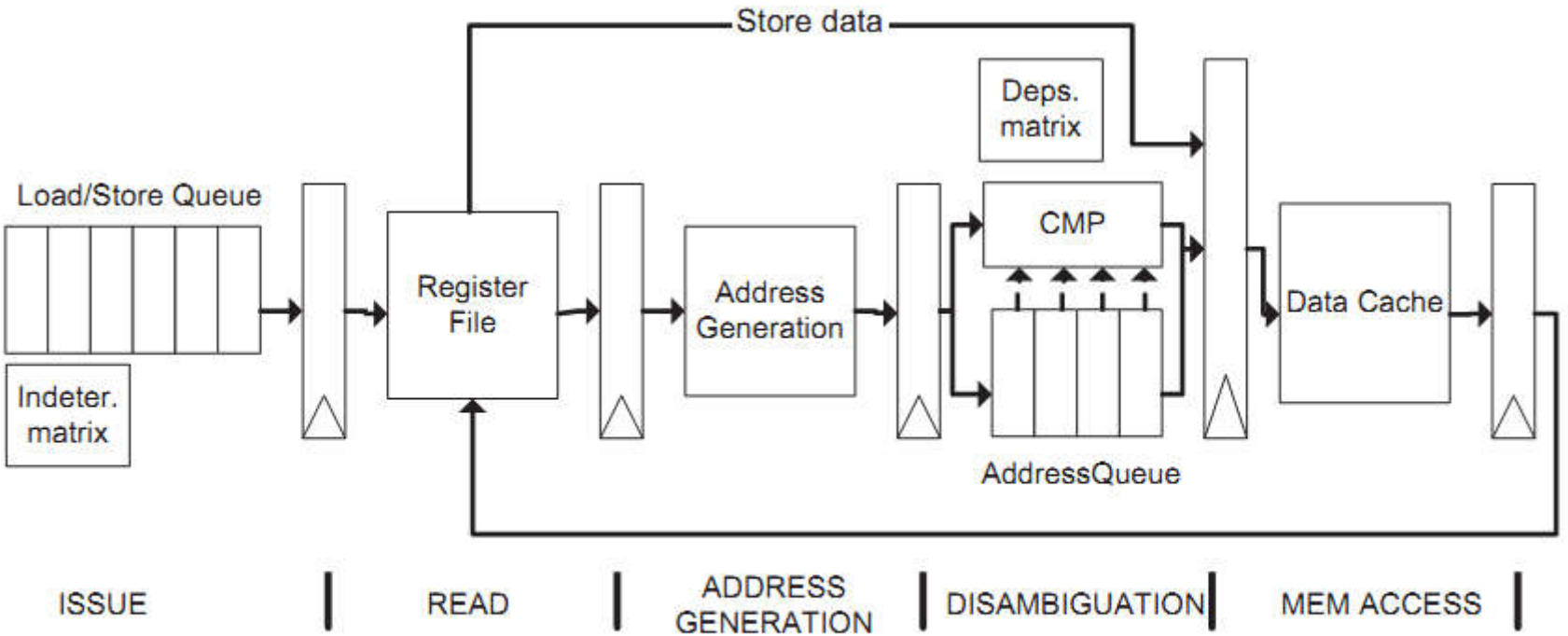


FIGURE 6.8: Schematics of the MIPS R10000 pipeline to implement the partial ordering memory disambiguation policy.

Load/store queue: 长度为16的队列，存储load/store指令，直到其操作数准备好。

Indetermination matrix: 16x16 矩阵（half)，每行每列代表队列中的存储器操作。当有存储器操作进入队列时，对应列置位。当存储器指令计算有效地址时，对应列复位。计算有效地址的条件之一，该操作对应行为全0

Dependency matrix: 16x16 矩阵，每行每列对应load/store队列的存储器操作。Load操作如果依赖于前面的store，则将该load操作的行中对应该store的列置位。Store操作更新存储器时，将复位该store操作对应的列。只有当load操作对应的行全0时，方可执行load操作

Address generation: 计算存储器操作的有效地址

Address queue:保存访问cache的loads/store操作的地址。如果是load操作，除了保存地址，还需要比较所有在该load操作之前的store的地址，如果匹配，则对dependency matrix对应位置位。

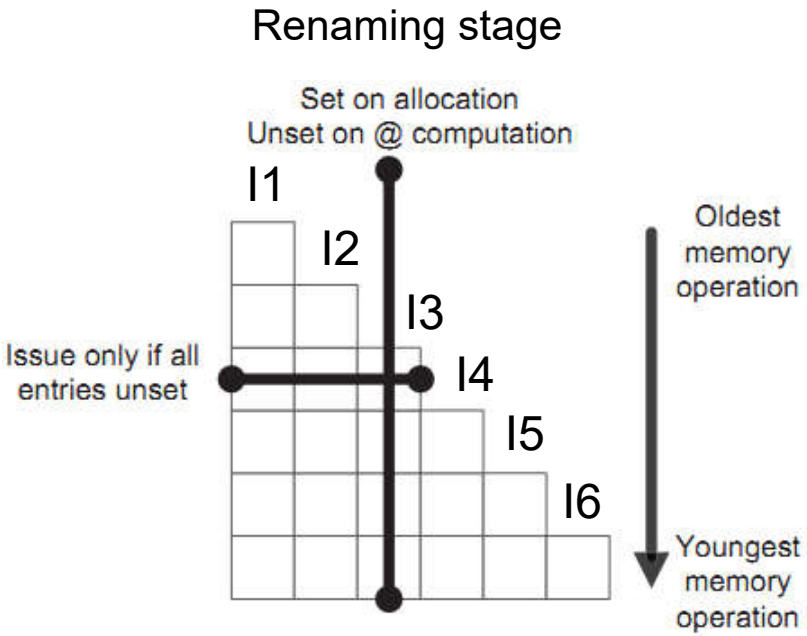


FIGURE 6.9: Example of a 6-entry indetermination matrix.

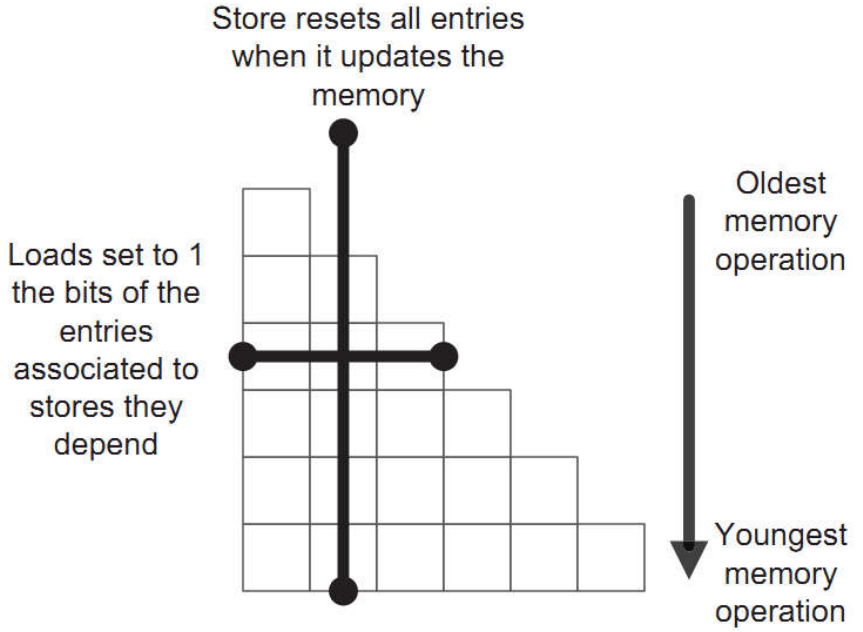


FIGURE 6.10: Example of a 6-entry dependency matrix.

Speculative Memory Disambiguation

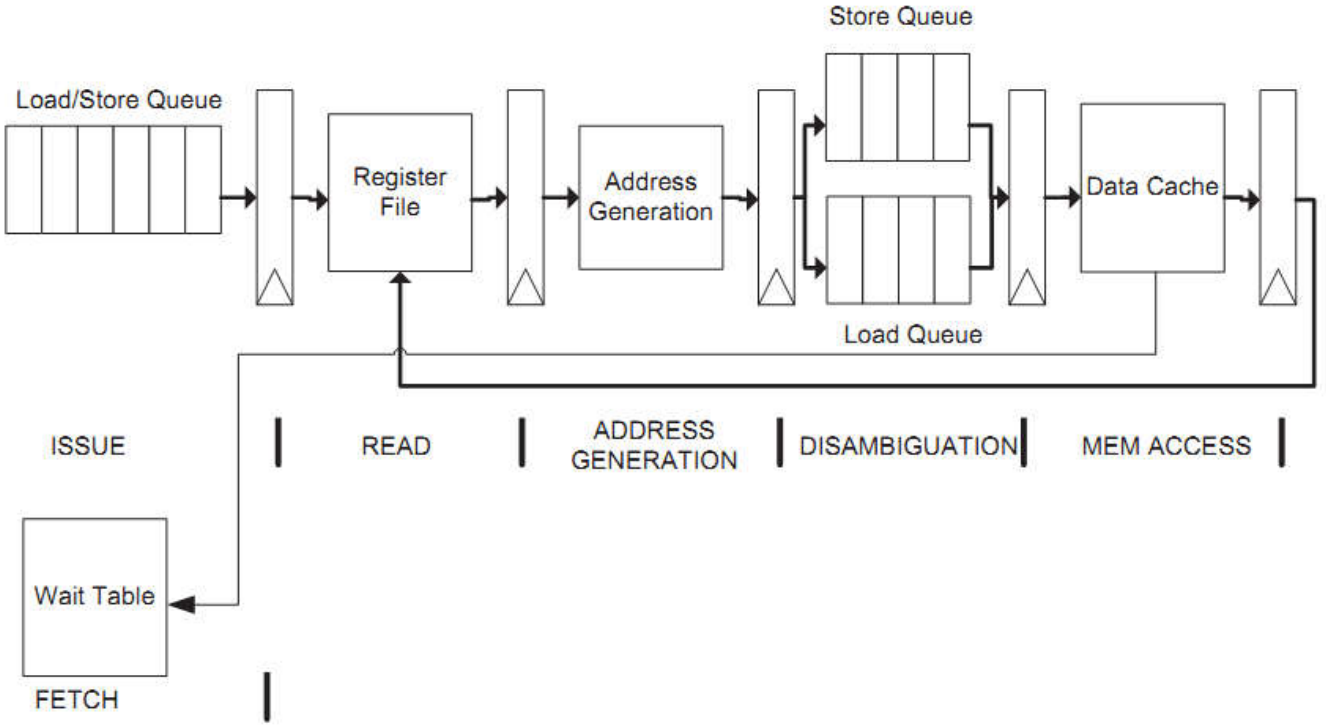


FIGURE 6.11: Schematics of the Alpha 21264 pipeline to implement a speculative memory disambiguation policy.

Load/Store Queue: 保存存储器操作的队列,直到可以计算有效地址

Load Queue: 该队列以程序序存储load操作的存储器物理地址。该队列有32项

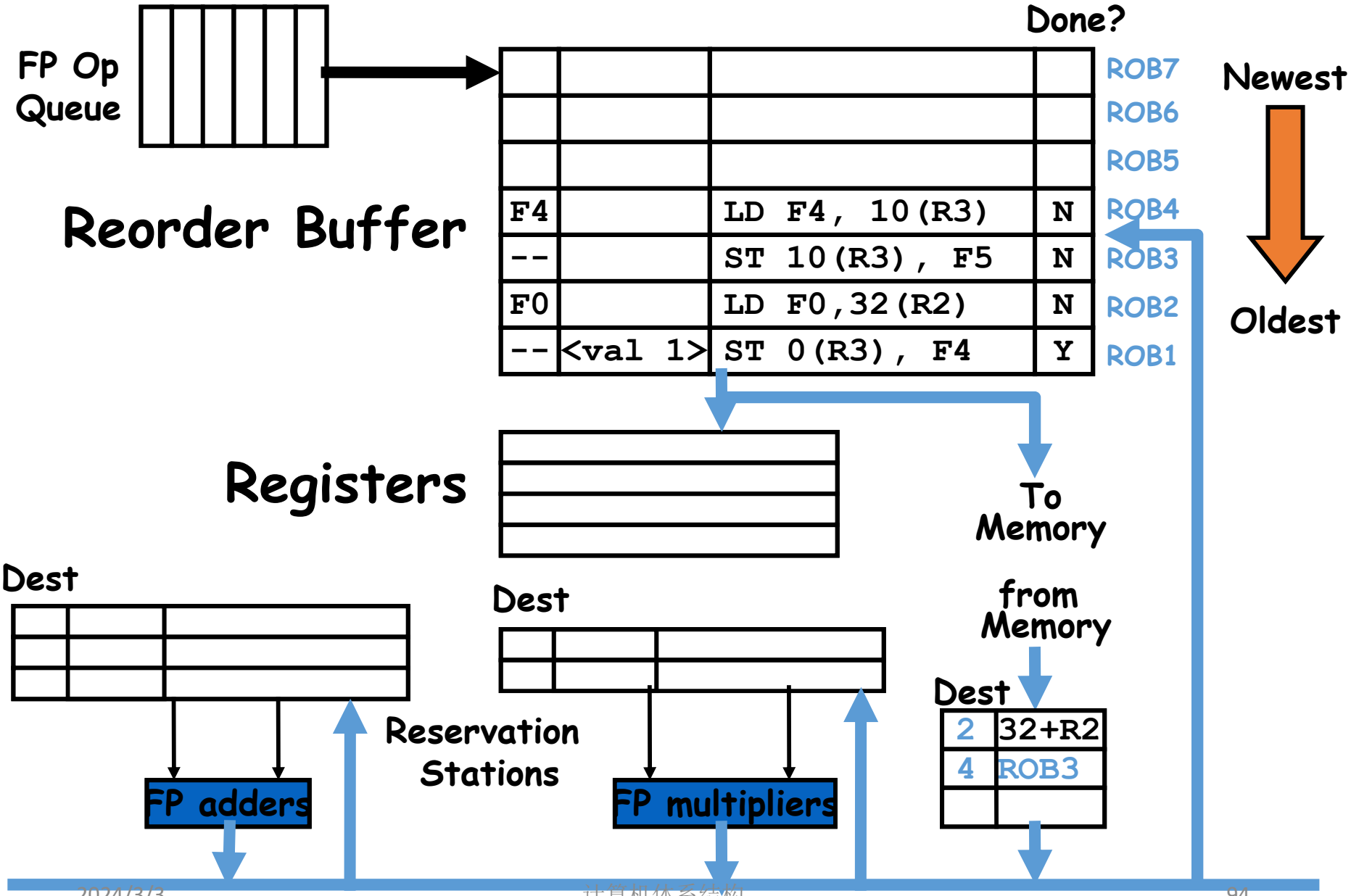
Store Queue: 存储store操作的存储器物理地址以及要存储的值。该对列32项

Wait Table: 具有1024 项，每项1位的表，由存储器指令虚拟地址索引. 当我们检测到一个load操作超越了与它相关的store操作时，该load对应项置1，取指部件读取该信息，将不会再提前发射出去。该表每隔**16384**周期复位一次，否则该表可能所有项均为1.

Memory Disambiguation -Review

- Reorder Buffer:
  - 提供了撤销指令运行的机制
  - 指令以发射序存放在ROB中
  - 指令顺序提交
- 分支预测对提高性能是非常重要的
  - 推断执行: 在控制相关还没有解决的情况下，就开始执行
  - 推断执行是利用了ROB撤销指令执行的机制
- Memory Disambiguation
  - Nonspeculative Memory Disambiguation
  - Speculative Memory Disambiguation

Memory Disambiguation:



Tomasulo+ROB summary

- 实际系统的实现非常相似
  - PentiumIII, PowerPC, 等
- 不足之处：
  - Too many value copy operations
    - Register File →RS→ROB→Register File
  - Too many muxes/busses (CDB)
    - Values are from everywhere to everywhere else!
  - Reservation Stations mix values(data) and tags (control)
    - Slow down max clock frequency

小结#1/2

- Reservations stations: 寄存器重命名，缓冲源操作数
  - 避免寄存器成为瓶颈
  - 避免了Scoreboard中无法解决的 WAR, WAW hazards
  - 允许硬件做循环展开
  - 不限于基本块(IU先行，解决控制相关)
- 贡献
  - Dynamic scheduling
  - Register renaming
  - Load/store disambiguation
- 360/91 后 Pentium II; PowerPC 604; MIPS R10000; HP-PA 8000; Alpha 21264使用这种技术

小结 #2/2

- 动态调度方案可以用硬件动态完成循环展开
  - 通过重命名机制来消除WAR和 WAW 相关
- Reorder Buffer:
  - 提供了撤销指令运行的机制
  - 指令以发射序存放在ROB中
  - 指令顺序提交
- 分支预测对提高性能是非常重要的
  - 推断执行是利用了ROB撤销指令执行的机制
- Superscalar 和VLIW: CPI < 1 (IPC > 1)

如何使CPI < 1 (1/2)

- 前面所述的各种技术主要通过减少数据相关和控制相关，使得CPI = 1 (CPI接近1）
- 是否能够使CPI < 1?
- 两种基本方法
- Superscalar:
  - 每个时钟周期所发射的指令数不定（1 －8条）
  - 由编译器或硬件完成调度
  - IBM PowerPC, Sun UltraSparc, DEC Alpha, HP 8000
  - 该方法对目前通用计算是最成功的方法
- 新的概念 Instructions Per Clock (IPC) vs. CPI

如何使 CPI < 1？ (2/2)

- (Very) Long Instruction Words (V)LIW:
  - 每个时钟周期流出的指令数（操作）固定 (4-16)
  - 由编译器调度，实际上由多个单操作指令构成一个超长指令
  - 目前比较成功的应用于DSP，多媒体应用
  - 1999/2000 HP和Intel达成协议共同研究VLIW
  - Intel Architecture-64 (Merced/A-64) 64-bit address
  - Style: “Explicitly Parallel Instruction Computer (EPIC)”

用于多发射处理器的五种主要方法

Common name	Issue structure	Hazard detection	Scheduling	Distinguishing characteristic	Examples
Superscalar (static)	dynamic	hardware	static	in-order execution	Sun UltraSPARC II/ III
Superscalar (dynamic)	dynamic	hardware	dynamic	some out-of-order execution	HP PA 8500, IBM RS64 III
Superscalar (speculative)	dynamic	hardware	dynamic with speculation	out-of-order execution with speculation	Pentium III/4, MIPS R10K, Alpha 21264
VLIW/LIW	static	software	static	no hazards between issue packets	Trimedia, i860
EPIC	mostly static	mostly software	mostly static	explicit dependences marked by compiler	Itanium

FIGURE 3.23 **There are five primary approaches in use for multiple-issue processors, and this table shows the primary characteristics that distinguish them.** This chapter has focused on the hardware-intensive techniques, which are all some form of superscalar. The next chapter focuses on compiler-based approaches, which are either VLIW or EPIC. Figure 3.61 on page 341 near the end of this chapter provides more details on a variety of recent superscalar processors.